

Anatomy of one simulation frame

symmetric-expansion · 180.42 ms

A pass-by-pass account of what happens inside one `advanceTo()` : what each stage computes, which part of Aanjaneya et al. 2017 it implements, and what the GPU actually does while it runs. Every number is measured from the capture below or read from the solver's own end-of-run audit. Nothing is modelled except where marked `[derived]` .

Machine **Apple M1 Max** · 98,304 threads

Lane **symmetric-expansion** · 299 advances

Grid **32 × 16 × 32** · fine factor 4

Frame wall **180.42 ms**

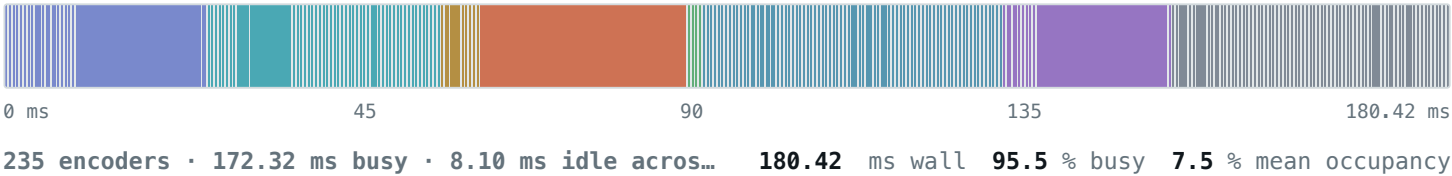
GPU busy **172.32 ms** · 95.5 %

Occupancy **7.5 % mean**

Bandwidth **3.1 GB/s** of ~78

Encoders **235** · 1 command buffer

THE FRAME — 235 METAL COMPUTE ENCODERS, WIDTH ∝ GPU TIME



THE THESIS

**The device is busy 96 % of the frame
and using 8 % of itself to do it**

Five encoders are 77 % of the 172 ms of attributed GPU time; the top ten are 83 %. One of the five — the pressure solve — is a single workgroup of 256 threads on a machine that holds 98,304, pinned to one of 4 GPU partitions while the other 3 measure 0.00 %. Select any segment above and the machine diagram below shows how much of the GPU it actually lit; each band's section then carries the *mechanism* — an interactive model of the launch, and the shader code that produces it.

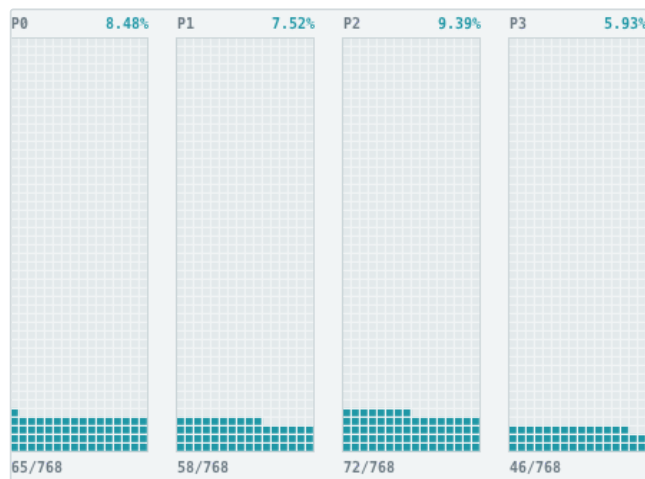
EVERY CAPTURE OF THIS LANE – 4 SHIPPING WALLS, OLDEST FIRST

3c84d76 · band 3		237.75 ms/adv
before either sampler was compiled		
c5061fc + a3adca9 · band 3		194.96 ms/adv
packed fine pages · compiled structured sampler		
18a7ad7 · band 3		159.26 ms/adv
compiled fine sampler · separable halo dilation		
c8c84ee · band 4		167.34 ms/adv

▲ different discretization from here down — band 4 made canonical — a scene change, measured here

The first three bars share interface band 3 and are directly comparable: **237.75 → 159.26 ms/advance, −33.0 %** across three commits, none of which touched the discretization. **The fourth is a different scene.** `c8c84ee` made interface band 4 canonical, which changes the refinement band, the row count and the page count — so its 167.34 ms is not a regression against 159.26 ms, it is a different measurement. **Everything else in this document describes that band-4 frame**, because it is what ships; where a before/after statement crosses the boundary it says which band each side came from.

MACHINE OCCUPANCY — WHOLE FRAME, MEAN

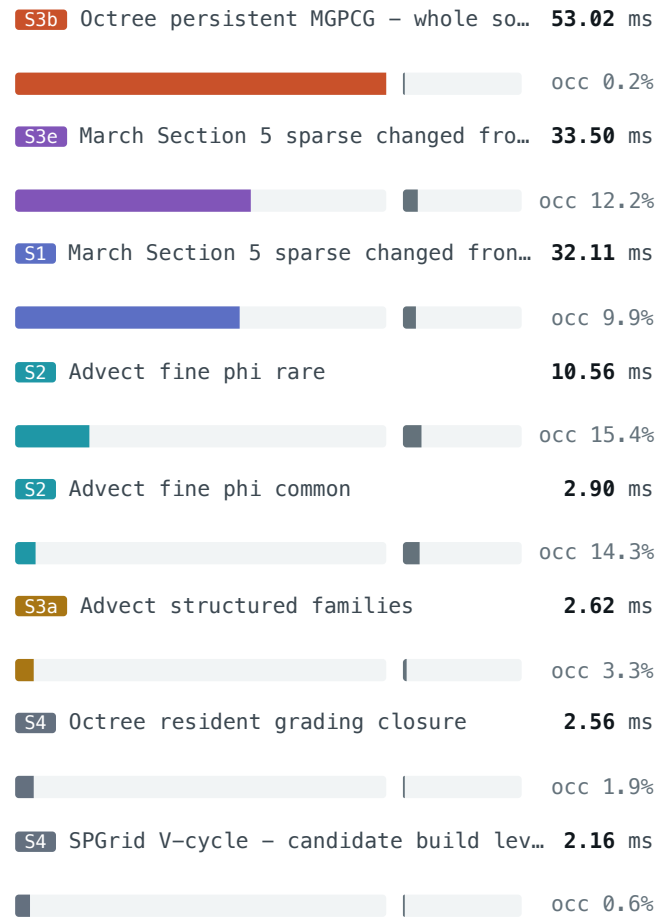


■ 7,712 resident threads = 7.85 % □ idle silicon

4 partitions × 768 slots × 32 = 98,304

One cell is one scheduler slot (32 threads). Apple exposes no per-wave trace, so this shows *how many* slots held a resident SIMD group, never which — the fill is the measured **Compute Occupancy** counter, laid out across the 4 partitions in their measured proportions.

TOP ENCODERS, REPRESENTATIVE ADVANCE



Bar fill is the encoder's share of the 172.50 ms attributed; the inner mark is its measured compute occupancy on the same scale as the machine diagram.

Measurement integrity — read this before quoting a number

PROFILING DISTORTION

1.04x

Untraced shipping wall **167.34 ms/advance**; label-isolated traced wall **174.08 ms**. Isolation splits 79 shipping passes into 235 labelled encoders and costs **4.0 %**: real, and worth subtracting before quoting a per-pass number as shipping cost.

ATTRIBUTION

100.0 %

225 exact stages, **0 composite**. Busy + gaps = frame wall exactly; nothing unattributed, nothing leaked in. 490 dispatches/advance in both runs.

COUNTER COVERAGE

82.4 %

GPU counters are device-wide; 82.4 % of our GPU time was uncontended (Codex (Service) took 43 ms). Per-task *time* is exact — it filters by process. Occupancy and ALU are directional.

THE 42 ADVANCES IN THE WINDOW



mean **182.06** ms min **171.50** max **204.60** repr. **180.42**

The fully-attributed decomposition is **one** advance. The frame is now flat to $\pm 9\%$ end to end and **one** task is not: the pressure solve, charted in S3b (44.20 – 61.27 ms). Structured advection used to be the other one at 1.9 – 2.8 ms; it is now **1.81 – 2.65 ms**, and that collapse of the range matters more than the mean — see S3a.

THE SUBJECT

One centred water block collapsing
across the smallest dyadic tank

A $1.6 \times 0.8 \times 1.6$ m closed tank with free-slip walls holding one centred $0.8 \times 0.4 \times 0.8$ m body (fill fraction $\frac{1}{8}$), seeded as four bricks. It is the smallest dyadic domain that leaves an equal one-brick gap to all four walls, which is why it is the project's D4 symmetry oracle: volume, velocity, pressure, topology and four-wall contact must stay exactly symmetric under the dihedral group of the horizontal plane.

DISCRETIZATION		PER ADVANCE, HOST SIDE		VA
Coarse simulation lattice		Command buffers / queue.submit		1
Maximum leaf size	32 cells	Compute passes (shipping)		75
Fine interface factor		Dispatches	490.0 – 313.0	indirect
Live pressure rows		copyBufferToBuffer	71.3	· 36.6
Power-diagram face slots		writeBuffer	31.0	· 6.4
Published descriptors	1,956 – 1	Bind groups created		2
Fine narrow band	8,612 pages of 13,05	MGPCG iterations	4 (budget 10, ceiling 1	
Fine page format	4	Relative residual	7.10×10^{-6}	
SPGrid V-cycle		Volume drift over 299 steps	-3.77×10^{-6}	
Sec 5 air-support graph	6,446	Rejected advances / tripwires		0
Allocation	181.7 MB authoritative			
Timestep				

The two descriptor counts are §6.1 measured in the wild

The paper defines exactly two topology cases — *cells with no coarser neighbour* (an 18-bit neighbour mask, 2^{18} possibilities) and *cells with no finer neighbour* (3 bits of child position plus 6 bits of coarse-neighbour direction, 2^9 possibilities), plus one discriminating bit. This scene publishes **1,956 of the first and none of the second**. At interface band 4 the refinement band is wide enough that every published cell has a finer neighbour somewhere in its ring, so the second case — the one that needs a child position and a coarse-neighbour direction — never arises. The adaptivity that pays for itself here is therefore not level-of-detail at all: it is that the 14,425 dry cells carry no rows. [At band 3 the same scene published 26 of the second case.]

The catalog those descriptors index is generated offline rather than populated on demand as the paper does: **8,083 configurations, 1,608 distinct descriptors, 14.18 MB, max 68 tetrahedra**, max face incidence 30, max 36 neighbour rows, worst float32 geometry error 5.7×10^{-8} . The paper reserves ~ 128 MB for its on-demand table; ours is smaller because it stores only the configurations the grading rule can actually produce.

A second, *derived* table now sits beside it: **28.9 MB** of compiled point-location structure — a selector $\times$ transform map, a tetrahedron adjacency graph, per-octant seeds and precomputed barycentric gradient rows. Nothing in it is new information; it is the same catalog rearranged so a query is a lookup rather than a search. That is the trade the frame just made — **+30.3 MB on the power authority**, against **−33 %** of wall clock across the three same-band captures that used it. Both samplers read it now — S3a and S2.

Eight bands, and a decomposition the paper would not recognise

The advance is one fixed sequence declared in `OCTREE_STEP_PROGRAM` and encoded into **one command buffer**, one `queue.submit` .

S1 ready-topology-flip

S2 surface-transport

S3a structured dynamics

S3b pressure solve

S3c projection

S3d fine settlement

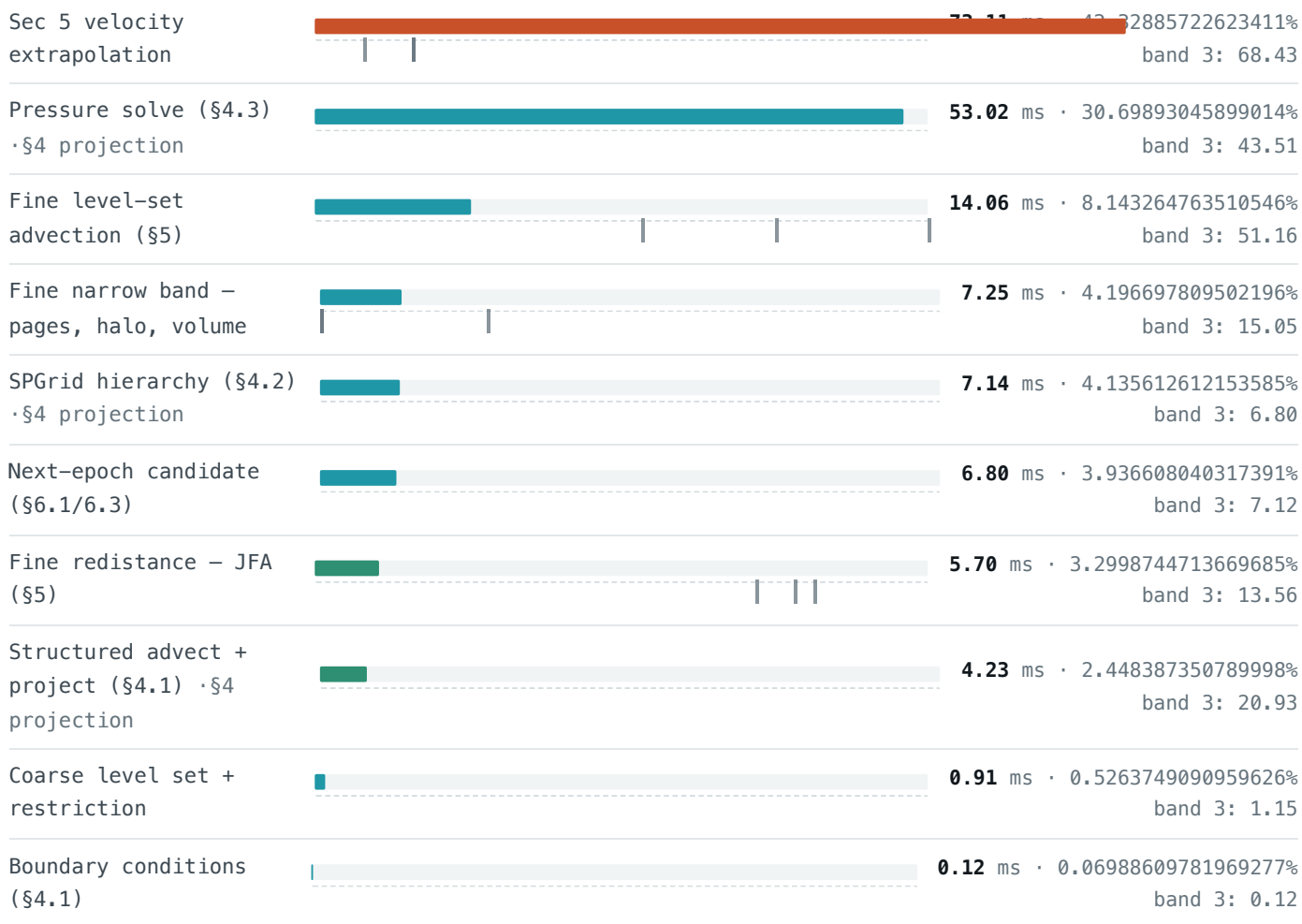
S3e Sec 5 support (settled)

S4 next-epoch candidate

BAND	WHAT IT DOES	ENCODERS	T (MS)	GPU MS	%	SHARE
S1	ready-topology-flip – Commit the epoch, rebuild the extended velocity field	19	0.0 – 38.3	37.67	21.8	
S2	surface-transport – Move the free surface	49	38.4 – 56.3	17.02	9.9	
S3a	structured dynamics – Advect the face velocities, apply forces, assemble the RHS	8	56.3 – 59.9	3.34	1.9	
S3b	pressure solve – 1,959 unknowns, one workgroup, 256 threads	1	59.9 – 112.9	53.02	30.7	
S3c	projection – Apply the gradient – 116 microseconds	4	113.0 – 113.2	0.12	0.1	
S3d	fine settlement – Redistance, correct volume, restrict to the octree	76	113.3 – 126.6	11.08	6.4	
S3e	Sec 5 support (settled) – The same 34 ms extrapolation, a second time	10	126.6 – 164.3	37.31	21.6	
S4	next-epoch candidate – Build and validate the whole next epoch, inactive	68	164.4 – 180.4	13.15	7.6	

BY SUBSYSTEM — MEASURED SHARE VS. AANJANEYA ET AL. TABLE 2

Top bar (solid) is this frame. Below it, the paper's three examples — Fig 2 twin-source sheet, Fig 3 rotating paddle, Fig 8 river canyon — as hollow marks on the same axis.



Vertical marks are the paper's own three examples on the same 0 – 32 % axis (Table 2, Fig 2 / Fig 3 / Fig 8). Its "Projection" line groups our pressure solve, structured operators and SPGrid setup — together 37.3 %, inside the paper's 30 – 49 % band. The grey figure is the previous capture under the identical label rule — measured at **interface band 3**, so read it as context, not as a delta.

Two differences carry the whole document

Velocity extrapolation is 42.3 % of our frame and 2.5 – 5 % of the paper's. Now that structured advection has collapsed, it is by a clear margin the single largest subsystem. The paper runs a serial CPU fast march over faces; we run a GPU frontier relaxation to a fixed point over 77,352 face unknowns — **twice per advance**. It is the one line that has not moved.

Reinitialization is 3.3 % of our frame and 23 – 26 % of the paper's. Replacing the paper's serial fast marching with a jump-flood closest-point transform is where we are

decisively ahead — the JFA kernels are the only ones in the frame at 51 % occupancy and 73 % ALU. Read that 3.3 % with the caveat in S3d: the conditional fallback ladder fired two captures back (8.28 ms) and not since, so the honest range is 3 – 8 %.

Projection as a whole (solve + operators + hierarchy = 37.3 %) lands squarely inside the paper's 30 – 49 % band, and 4 PCG iterations sits inside its "satisfactory convergence within 6 – 10".

The eight bands in execution order

Each band: what it is for, how the paper's method is realised, and what the GPU did.

Timings are the representative advance unless a mean is given. Clicking a band's pass table row pins that encoder in the ribbon above.

S1 Commit the epoch, rebuild the extended velocity field

37.67 ms 21.8 % of frame 19 encoders t 0–38 ms ready-topology-flip

GOAL

The active topology epoch is immutable for a whole advance. Everything about the *next* epoch — refined octree, power descriptors, §6.3 row coefficients, structured velocity bank, boundary coefficients, SPGrid level sets — was built at the *previous* advance's tail and cross-validated there. S1 is the single seam where that candidate becomes accepted, atomically across seven independent authorities, and where the §5 extended velocity field is rebuilt against the newly accepted geometry so everything downstream can sample velocity anywhere, including in air. A poisoned candidate leaves the gate retaining the current epoch. Zero rejections in 240 advances.

HOW THE 2017 PAPER IS REALISED

§4.2 pyramid of sparse uniform grids. The commit publishes each level's active/ghost cell sets. SPGrid V-cycle · publish validated exact level deltas is the moment the six-level pyramid the preconditioner walks becomes visible to the solver — as an exact *delta* against the previous generation, not a rebuild.

§6.1 topology encoding. Commit power descriptor publication and Commit power topology publication flip the descriptor words and the catalog-resolved PowerRowMetric (19 stencil coefficients, face areas, normals, inverse distances) from the candidate bank to the accepted bank.

“To extrapolate velocities outside the liquid region we first interpolate velocities from power faces to regular octree faces and use an approach similar to fast marching, this time operating on faces instead of cells and copying over the velocity value from the face closest to the free surface. ... Subsequently, we interpolate back velocities from regular octree faces to all power faces.”

Aanjaneya et al. 2017, §5

That is exactly passes 10 – 19: Extrapolate structured ordinary faces → March ... to a fixed point → Reconstruct Section 5 air-support vectors.

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



The commit itself is free. Eight authorities flip in $60\ \mu\text{s}$ of single-workgroup dispatches. The design goal — that an epoch flip cost nothing at the seam because all the work happened a step earlier — is met exactly.

The march is 85 % of the band. Data layout: the air-support arena holds a face-value bank `faceA` and a proposal bank `faceB`, each indexed `row × 12 + 4·axis + quadrant` — 12 face slots per owner row (3 velocity axes × 4 power-face quadrants), 6,446 rows, **77,352 slots** — plus an atomic `faceFrontier` control block holding three per-axis queue pairs and a generation-marked dedup word per face.

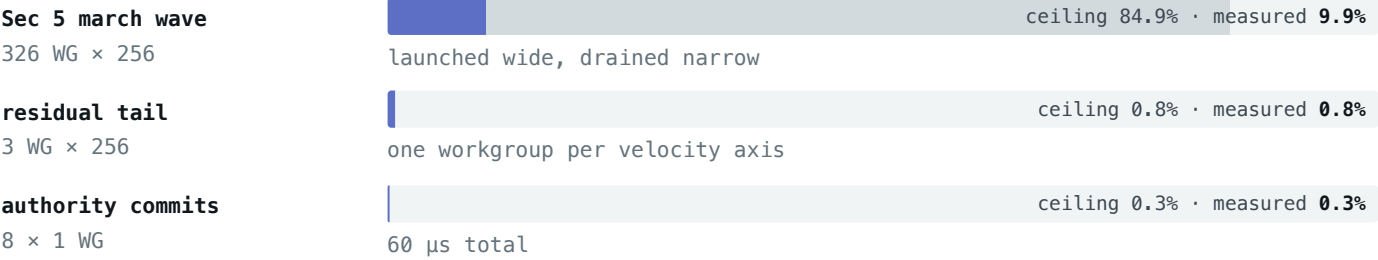
It is a monotone frontier relaxation, not a fast march. **12 wide prefix waves**, all four dispatches of every wave inside *one* compute pass: `expand` (256 lanes; each lane owns a packed `(axis, queue slot)` pair, walks up to 30 incident rows from the catalog adjacency and appends 4 quadrant destinations each with an `atomicExchange dedup`) → `relax` (each surviving destination runs the 30×4 closest-face gather into `faceB`) → `commit` (`faceB ≠ faceA` → write back and enqueue) → `advance` (**one workgroup, one lane**, rotating the three queue pairs). Then a residual tail — **3 workgroups**, one per velocity axis — loops to a fixed point inside the kernel. `structuredAirSupportMarchDepth: 0` says the tail did no extra waves.

Every wave dispatches the same fixed grid. The per-wave indirect record is only re-authored on the factor-1 path (`p.fineFactor == 1u` inside `advanceAirSupportChangedFrontier`); at fine factor 4 all 36 `expand/relax/commit` dispatches read one record sized to the *published face count* — 77,352 slots, **326 workgroups, 83,456 threads, 85 % of residency**. The frontier shrinks; the launch does not. Lanes past the live queue count return on their first branch, so measured occupancy time-averages to **9.9 %**, and each of the 48 dispatch boundaries pays full launch-and-drain latency on a nearly-empty grid.

The gather is not the cost. The source carries an A/B result for exactly this: running the 30×4 candidate scan *twice* — result-preserving, zero tripwires — moved the pass $39.68 \rightarrow 43.52\ \text{ms}$, so one complete scan is $\sim 3.84\ \text{ms}$ against a $3.65\ \text{ms}$ A/A floor. **The scan is $\sim 10\%$ of the pass; the other $\sim 36\ \text{ms}$ is the frontier machinery around it.** No partial optimisation of the gather can pay.

32.1 ms to fill 53,436 unseeded face values is $\approx 601\ \text{ns}$ per face (77,352 slots less 23,916 seeds). And it runs a second time in S3e — the same 34 ms again.

LAUNCH SHAPE — THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT



Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident [derived from workgroup count × workgroup size] ; the coloured bar is the measured time-averaged Compute Occupancy counter.

LAUNCHED WIDE, DRAINED NARROW a provisioned-capacity grid over a work set that is orders smaller

32.11 ms of the band's 37.67 is one encoder, and it dispatches 326 workgroups over all 77,352 face slots on every one of its 12 waves. The frontier shrinks; the launch does not. 85 % residency asked for, **9.9 %** measured.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked [derived] .

A sparse frontier dispatched on a dense, fixed grid

March Section 5 sparse changed frontier to a fixed point

Velocity has to exist everywhere the level set might backtrace into — including inside air — so \$5 extends the liquid face velocities outward. The paper fast-marches this serially on the CPU. We relax a monotone frontier on the GPU: a queue of faces that *changed* in the previous wave expands through the reciprocal incidence graph, each destination re-runs a closest-face gather, and a face that moves re-enqueues itself. Twelve host-encoded waves, then a 3-workgroup residual tail that loops to a fixed point in-kernel.

The algorithm is sparse. **The launch is not.** The per-wave indirect record is only re-authored on the factor-1 path; here every wave dispatches one fixed record sized to the whole published face space. Drag the frontier size below and watch what that costs.

EVERY WAVE DISPATCHES 326 WORKGROUPS OVER ALL 77,352 FACE SLOTS

0 1 2 3 4 5 6 7 8 9 10 11 T prefix wave → expand → relax → commit → advance, 4 dispatches, one compute pass
83,456 lanes launched · 4,000 do work · 79,456 return on their first branch



```
if (local >= current) { return; }  
- expandAirSupportChangedFrontier, and the same guard in relax and commit  
residency this dispatch actually asks for: 4.07 % of the machine
```

dispatch 326 WG × 256 useful lanes 4.79 %

measured time-averaged occupancy 6.7 % (S1) / 11.1 % (S3e) 48 dispatch boundaries in one encoder

The 326-workgroup grid, the 77,352 slots and the 12 waves are measured. The frontier size is yours to set — per-wave frontier counts were not instrumented in this capture. Whatever you choose, the launch does not change: that is the point.

THE CODE THAT PRODUCES IT

the 12 waves expand relax + commit the gather (measured)

lib/webgpu-octree-air-velocity-support-gpu.ts · from line 1392 TS · ... marks an elision

```
1  const frontierWaves = octreeAirSupportParallelFrontierWaves(); // = 12
2  for (let wave = 0; wave < frontierWaves; wave += 1) {
3      const waveOffset = indirectFrontierGate ? 0 : 48; // gate is false at
        fineFactor 4
4      pass.setPipeline(this.pipelines.expandAirSupportChangedFrontier!);
5      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
6      pass.setPipeline(this.pipelines.relaxAirSupportChangedFrontier!);
7      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
8      pass.setPipeline(this.pipelines.commitAirSupportChangedFrontier!);
9      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
10     pass.setPipeline(this.pipelines.advanceAirSupportChangedFrontier!);
11     ...
12     pass.dispatchWorkgroups(1); // singleton: rotate the
        queues
13 }
14 // residual tail: exact, unbounded, one workgroup per velocity axis
15 pass.setPipeline(this.pipelines.marchAirSupportFacesChangedFrontier!);
16 pass.dispatchWorkgroups(3);
```

Record 48 is the whole face space. It is authored once, as `dispatchFor(s(29u), 256u)` where `s(29)` is the published face count `77,352 → 326 workgroups`. Wave 11 launches exactly as wide as wave 0. All 48 dispatches sit inside a single compute pass, which is why the profiler reports them as one 32.1 ms encoder — and why there are 48 device-wide serialisation points inside it.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
March Section 5 sparse changed frontier to a fixed point – topology-commit		33.06	30.92	36.57	32.11
SPGrid V-cycle – publish validated exact level deltas		1.69	1.64	1.96	1.65

HEADROOM IN THIS BAND

2. Halve the Sec 5 marches ≈33 ms shared with S3e — Now unambiguously the biggest structural redundancy left: the same extrapolation, twice per advance. Share the extended field between the topology-commit and settled-fine sites, or make the changed-frontier seeding actually sparse on this scene. The paper needs the extended field to exist once per surface update; we build it twice.
5. Fix the wide-launch / narrow-drain shape of the march waves shared with S3e — Part of the same 33 ms. 12 host-encoded barrier waves over a provisioned-capacity grid, when the live frontier is a few thousand faces, is the textbook case for a persistent multi-

workgroup kernel with an on-GPU queue. The source already measured that the gather inside it is only $\sim 10\%$ of the pass.

EVERY ENCODER IN S1, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
18	March Section 5 sparse changed frontier to a fixed point - topology-commit	32.106	85.2	9.9	22.3	1.43
9	SPGrid V-cycle - publish validated exact level deltas	1.650	4.4	0	0	4.00
11	Publish structured air-support identities - topology-commit	0.993	2.6	36.7	23.1	1.09
12	Close sparse fine air-support demand - topology-commit	0.924	2.5	6.9	10	-
19	Reconstruct Section 5 air-support vectors - topology-commit	0.917	2.4	6.4	11.7	1.55
17	Extrapolate structured ordinary faces and reconstruct support vectors - topology-commit	0.491	1.3	2	2.6	-
14	Canonicalize footprint air-support identities - topology-commit	0.262	0.7	3.1	4.2	1.54
16	Resolve and hash live adaptive air owners - topology-commit	0.171	0.5	24.4	19.9	-
13	Emit sparse fine air-support demand - topology-commit	0.039	0.1	-	-	-
15	Publish canonical air-support candidates - topology-commit	0.027	0.1	-	-	-
10	Initialize structured air-support publication - topology-commit	0.026	0.1	-	-	-
3	Commit accepted topology row identities and pressure seed	0.013	0.0	-	-	-
5	Commit power descriptor publication	0.011	0.0	-	-	-
6	Commit power topology publication	0.010	0.0	-	-	-
4	Publish ready owner-page generation	0.008	0.0	-	-	-
1	Open coupled topology ready-commit gate	0.005	0.0	-	-	-
8	Accept structured boundary publication	0.005	0.0	-	-	-
7	Accept structured publication	0.005	0.0	-	-	-
2	Prepare exact accepted topology row commit	0.004	0.0	-	-	-

s2 Move the free surface

17.02 ms 9.9 % of frame 49 encoders t 38–56 ms surface-transport

GOAL

Advect the $128 \times 64 \times 128$ narrow-band level set and the coarse octree level set forward by Δt with the previous advance's projected, closest-point-extended velocity; rebuild the narrow band's page topology around the new interface; hand the transported generation to the projection stage.

HOW THE 2017 PAPER IS REALISED

“we divide this time step by m , where m is the factor by which the fine level set resolution is higher than the effective resolution of the octree. Next, we take m steps (of forward Euler) backwards along the velocity field ... After the m^{th} step we interpolate the level set, and assign its value back to the starting cell of the trajectory.”

Aanjaneya et al. 2017, §5 – Level set advection

Plan GPU-resident fine transport substeps computes `required = max(max(1, segments), ceil(v_max*dt/h_fine))` with `segments = fineFactor = 4`. Measured `transportMaximumDisplacementFineCells: 1`, so $m = 4$ — exactly the paper's rule, with a CFL escape hatch that did not fire.

Velocity interpolation on the power dual mesh. Each substep calls

`transitionSample()`, which locates the owning power cell and — if it is not the regular case — walks its catalog tetrahedra (up to 68), forms barycentric weights against the local Delaunay tetrahedralisation and blends four cell-centre velocities. That is §5's "trilinear interpolation inside cubic cells and barycentric interpolation on tetrahedra" plus §6.2's retrieval of local Delaunay tetrahedralizations from the byte-encoded vertex table.

The trace is the paper's, exactly. At `p.segments != 1` — which is every fine-factor-4 lane, including this one — each of the m stages is a single forward-Euler step: `x -= dt*transitionSample(x)`. A BFECC ladder (RK2 predictor → Reverse factor-1 predicted `phi` → bounded MacCormack correction) exists in the same file but `coarsePhiBFECCEnabled` requires `fineFactor === 1` and an explicit opt-in, so it compiles out here and publishes no encoders — you will not find one in Appendix A. The paper's Limitations names higher-order transport as future work; on this lane we have not taken it.

“we first allocate all pages corresponding to blocks that either contain the interface, or lie in the 1-ring of a block that contains the interface. After this step, cells that lie within the narrow band can be safely marked in parallel without any data hazards.”

This dilation is exactly that step — with a **4-ring**, not a 1-ring, because the redistance band plus the maximum backtrace need the extra collar. It is now performed as a separable X/Y/Z box sweep rather than a cubic scatter; the membership set is unchanged, which the audit confirms (8,612 pages, 1,583 of them interface, both identical to the scatter's).

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE – WHAT THE GPU ACTUALLY DID

Transport is 13.46 ms, 8 % of the frame. One workgroup of **64 lanes per fine page**; each lane strides the page's 64 samples, so one lane $\approx$ one fine cell. Addressing is $(\text{pageId} \ll 6) \mid \text{lz} \ll 4 \mid \text{ly} \ll 2 \mid \text{lx}$ — a pure shift, no division. The page is now **one u32 per sample: 256 B, down from 1024**; the mechanism below shows the bit layout and what it cost.

Per advected sample: **4 substeps** $\times$ **transitionSample** — each a power-cell owner lookup and, off the regular case, an **11-step compiled walk** of the local tetrahedral fan — then **one terminal 8-corner sampleFine** trilinear gather. The audit reports **398,484 samples processed** per step, so 13.46 ms is **34 ns per transported sample, ≈ 7 ns per interpolation**. The walk replaced a scan of up to 68 tetrahedra, and it is *unrolled* rather than looped — see the mechanism for why that distinction is the whole point in this kernel.

Occupancy 15.4 – 14.3 %, ALU 38 – 27 %. With 8,612 pages nothing is starved, so this is register-pressure and divergence limited — the frame's one heavy kernel doing genuinely dense arithmetic. **The common/rare split still costs more than it saves:** a page is **rare** if *any* of its 64 samples has $\phi \geq 0$ or is not VALID, so around a moving surface nearly every page is rare — 10.56 ms against **common**'s 2.90 ms.

The halo dilation stopped being cubic. It was 8.15 ms of near-pure atomic traffic — $1,851 \text{ seeds} \times 729 \text{ offsets} = 1,349,379$ idempotent **atomicOr**s to produce 9,518 pages, $142 \times$ write amplification. A Chebyshev ball is a Cartesian box, so it separates: seed once, then sweep X, Y, Z. **0.08 ms across 5 encoders**, membership bit-identical, and the frame lost a pass because the storage-to-indirect fence that sized the old pair grid went with it.

LAUNCH SHAPE — THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT

Advect fine phi 8,612 WG × 64	ceiling 100.0% · measured 14.9% saturating launch — register/divergence bound
Seed · dilate ×3 · commit halo 64 WG × 256, 5 passes	ceiling 100.0% · measured not sampled separable box dilation — no atomic pair grid
workset build 1 WG singletons	ceiling 0.3% · measured 0.3% classify → reduce → scan → publish

Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident `[derived from workgroup count × workgroup size]` ; the coloured bar is the measured time-averaged `Compute Occupancy` counter.

ACTUALLY HEALTHY dense arithmetic, or a launch whose work was deleted rather than widened

Transport is the frame's one genuinely dense kernel — 8,612 workgroups, nothing starved, 15 % occupancy at 27 – 38 % ALU, and its sampler is now compiled and branch-free. The **halo dilation** that used to be the frame's only well-fed launch — 8.15 ms of atomic scatter at 79 % occupancy — is 77 μ s across 5 separable passes. The band went 59.55 → 17.02 ms.

MECHANISMS

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked `[derived]` .

One workgroup per page, one lane per fine cell, $m = 4$ backtraces each

Advect fine phi common / rare

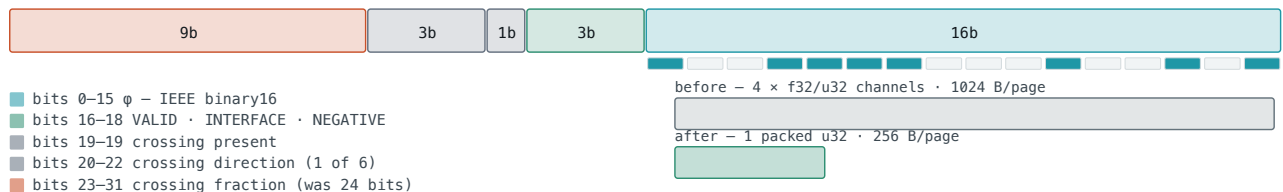
This is the only heavy kernel in the frame doing genuinely dense arithmetic, and structurally it is the healthiest launch: 8,612 workgroups is far more than the machine can hold at once, so nothing is starved. It still measures 15 % occupancy, because each lane holds a lot of live state and the work per lane is wildly uneven.

The §5 rule is that a fine level set at $m \times$ the octree resolution must take m sub-steps so a characteristic never skips a coarse cell. The planner computes m on the GPU and this scene lands on the floor: `segments = fineFactor = 4` beats the CFL term, which measured 1.

Two things changed under it in successive captures. The page these lanes stride was repacked — four separate 32-bit channels became one — and then the sampler each lane calls **11 times per sample** was compiled. The walk tabs carry the second story, and the unrolling in them is the part that is specific to this kernel.

ONE U32 PER FINE SAMPLE — 1,024 B/PAGE BECAME 256 B

bit 31 ← the one stored word → bit 0



ϕ round-trips to $-6.0005e-3$ m – error 0.004 % of one fine cell

ϕ in **-0.0060** m ϕ out **-0.006001** page **1,024 → 256 B** resident band **13.4 → 3.3 MB**

fine level-set arena **49.9 → 45.9 MB** (band 3 → 4) volume drift **$-1.14 \times 10^{-7} \rightarrow -3.77 \times 10^{-6}$**

Field offsets and widths are read from `encodeFineLevelSetPackedFlags`; the ϕ round-trip is the real binary16 conversion, run live. Resident totals are the 13,056-page arena. The fine cell here is 0.0125 m.

THE CODE THAT PRODUCES IT

the packed sample	deleting the neighbour tables	choosing m	the m-step trace	
the compiled walk	why it is unrolled	common vs rare		

lib/fine-levelset-packed-sample.ts · from line 80 TS

```
1  /** Persistent flags are VALID, INTERFACE and NEGATIVE. The former KNOWN and
2   * TRIAL bits belonged to the retired serial march. Nine fraction bits leave
3   * a worst-case cached crossing displacement below 0.001 fine cells. */
4  export function encodeFineLevelSetPackedFlags(flags: number): number {
5      const word = flags >>> 0;
6      const closest = word >>> 5; // the cached interface crossing
7      const present = closest !== 0 ? 1 : 0;
8      const direction = (closest >>> 24) & 7; // which of 6 axis directions
9      const fraction = closest & 0x00ff_ffff;
10     const fraction9 = Math.min(0x1ff, (fraction + 0x3fff) >>> 15); // 24 bits -> 9
11     return ((word & 1) | (word & 2) | ((word >>> 2) & 4)
12         | (present << 3) | (direction << 4) | (fraction9 << 7)) >>> 0;
13 }
14 export function packFineLevelSetSample(phi: number, flags: number): number {
15     return (float32ToFloat16Bits(phi) | (encodeFineLevelSetPackedFlags(flags) << 16)) >>> 0;
16 }
```

One u32 per sample: **binary16 ϕ in the low half, sample state in the high half**. The three persistent flags survive; KNOWN and TRIAL — state belonging to the serial fast march we no longer run — are gone, and the closest-point crossing is requantised from 24 bits to 9. The JFA ping-pong identities left the page ABI entirely and live in transient scratch. Net: `FINE_LEVELSET_CHANNELS` 4 $\rightarrow$ 1, page 1,024 B $\rightarrow$ **256 B**.

A Chebyshev ball is a box, and a box separates

Seed · dilate X/Y/Z · commit recurring fine-band halo

Before the narrow band can be marked in parallel, the pages it will need have to exist. The paper allocates blocks containing the interface plus their 1-ring. We need more — the redistance ladder and the maximum backtrace both reach outside the interface page — so the dilation radius is derived from those and lands at 4.

Until this capture that dilation was a **cubic scatter**: one invocation per (seed, halo offset) pair over a $(2r+1)^3$ volume, each issuing one idempotent `atomicOr`. 1,851 seeds $\times$ 729 offsets = **1,349,379 atomic writes to produce 9,518 pages**. It cost 8.15 ms and was the frame's only well-fed launch, spending 79 % occupancy on almost pure write traffic.

The replacement rests on one observation: **the Chebyshev ball this dilation wants is a Cartesian box**, and a box dilation separates into independent per-axis passes. Seed each key once, then sweep X, then Y, then Z. Membership is identical — the audit confirms the same 8,612 pages, the same 1,583 interface pages — but the work stops being cubic in the radius and stops being atomic. Move the radius slider below and watch the two curves diverge.

DILATION RADIUS: CUBIC IN THE SCATTER, LINEAR IN THE SWEEP

cost by dilation radius — solid = retired cubic scatter, hollow = shipped separable sweep

radius 1 ($3^3 = 27$)		0.3 ms → 0.03 ms
radius 2 ($5^3 = 125$)		1.4 ms → 0.04 ms
radius 3 ($7^3 = 343$)		3.8 ms → 0.06 ms
radius 4 ($9^3 = 729$)		8.2 ms → 0.08 ms
radius 5 ($11^3 = 1331$)		14.9 ms → 0.09 ms
radius 6 ($13^3 = 2197$)		24.6 ms → 0.11 ms

1,154,007 atomic writes → 458,752 plain reads across 5 passes — and the reads are unit-stride along one axis

radius 4 bricks scatter **1,154,007** atomic writes sweep **458,752** reads ratio **2.5×**

measured at radius 4 **8.15** → **0.08** ms

Both curves are calibrated on a measured point — the scatter on its own 8.15 ms at radius 4 (1,583 seeds $\times$ 729 offsets), the sweep on this capture's 0.08 ms over 5 encoders. Everything off radius 4 scales those measured costs by the operation count [derived]. Membership is identical at every radius; only the work is different.

THE CODE THAT PRODUCES IT

where the 4 comes from

the cubic scatter (retired)

the separable dilation

the four passes

lib/webgpu-octree-fine-levelset-topology.ts · from line 2035

WGSL · reformatted whitespace only

```
1 fn recurringDilationBrickRings() -> u32 {
2   let landing = transportDelta[7] + params.interpolationSupportFineCells;
3   let required = max(params.redistanceBandFineCells, landing);
4   return (required + params.brickResolution - 1u) / params.brickResolution
5     + params.safetyBrickRings;
6 }
7 fn recurringHaloRadius() -> u32 { return control[6]; }
```

Not an arbitrary constant: the radius is the redistance band and the transport landing distance, converted to bricks, plus a safety ring. It is load-bearing — the 1-ring the paper uses would leave the JFA ladder without pages to flood into. The *shape* of the dilation was what was wrong, never its reach, and the shape is what changed.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
Advect fine phi rare		10.53	10.27	10.93	10.56
Advect fine phi common		2.90	2.81	3.12	2.90
Finalize global fine page delta		1.22	1.19	1.28	1.25

HEADROOM IN THIS BAND

4. Split the fine transport's common/rare classes on something that discriminates ≈ 3 ms — A page is rare if *any* of its 64 samples is air or invalid, so around a moving surface nearly every page is rare and the split buys almost nothing — 10.56 ms in the expensive class against 2.90 ms in the cheap one. A predicate that actually separates cost (fraction of transition-cell lanes, say) would let the cheap class take a genuinely cheaper path.

EVERY ENCODER IN S2, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
29	Advect fine phi rare	10.560	62.0	15.4	37.9	1.02
28	Advect fine phi common	2.900	17.0	14.3	27.1	1.15
51	Finalize global fine page delta	1.250	7.3	1.6	0.7	1.14
67	Fine JFA – refresh transported closest-point codes	0.777	4.6	–	–	–
32	Commit structured fine phi and phase delta	0.250	1.5	19	22.8	1.01
22	Plan GPU-resident fine transport substeps	0.184	1.1	–	–	–
20	Maintain compact octree fine seeds	0.140	0.8	0.2	0.2	3.00
45	Scan and compact recurring fine-band logical identity	0.137	0.8	–	–	–
39	Publish recurring sparse fine band (compact seed classification and rank)	0.069	0.4	–	–	–
21	Publish deterministic fine-seed brick residency	0.066	0.4	–	–	–
49	Classify exact changed-key dirty and support pages	0.059	0.3	–	–	–
23	Classify direct structured fine transport blocks	0.049	0.3	5.7	16.5	1.04
55	Gather compact global fine work A/B payloads	0.048	0.3	29.9	18.3	–
57	Initialize added global fine samples	0.045	0.3	–	–	–
46	Classify exact global fine page delta	0.038	0.2	–	–	–
59	Gather all compact global fine adjacency	0.031	0.2	–	–	–
54	Gather compact global fine flags/phi payloads	0.030	0.2	–	–	–
44	Commit recurring fine-band halo	0.029	0.2	–	–	–
56	Snapshot exact global fine rollback pages	0.027	0.2	–	–	–
63	Fine-to-coarse restriction – publish	0.026	0.2	–	–	–
68	Power coarse level set – persistent schedule	0.025	0.1	0.4	0.9	2.53
25	Scan direct structured fine transport workset groups	0.024	0.1	–	–	–
24	Reduce direct structured fine transport workset blocks	0.017	0.1	–	–	–
35	Publish structured fine transport delta	0.017	0.1	–	–	–
34	Reduce structured fine transport delta blocks	0.016	0.1	–	–	–
64	Fine JFA – publish support mask	0.016	0.1	–	–	–
43	Dilate recurring fine-band halo axis 2	0.015	0.1	–	–	–
30	Reduce structured fine transport status blocks	0.014	0.1	–	–	–
31	Publish structured fine transport status	0.013	0.1	–	–	–
42	Dilate recurring fine-band halo axis 1	0.013	0.1	–	–	–
41	Dilate recurring fine-band halo axis 0	0.012	0.1	–	–	–
52	Publish physical power-cell volumes	0.011	0.1	–	–	–
50	Compact exact dirty and support pages	0.011	0.1	–	–	–
53	Publish exact post-redistance fine summary keys	0.008	0.0	–	–	–
47	Compact sorted global fine changed keys	0.008	0.0	–	–	–
40	Seed recurring fine-band halos	0.008	0.0	–	–	–
61	Fine-to-coarse restriction – prepare	0.007	0.0	–	–	–
65	Power coarse level set – publish exact dispatch	0.007	0.0	–	–	–
27	Compact direct structured fine transport worksets	0.007	0.0	–	–	–
48	Prepare global fine page delta expansion	0.007	0.0	–	–	–
26	Publish direct structured fine transport worksets	0.007	0.0	–	–	–

#	ENCODER	MS	%	OCC	ALU	PLC
36	Compact structured fine topology delta	0.007	0.0	–	–	–
38	Clear recurring fine-band identity mask	0.007	0.0	–	–	–
58	Initialize added global fine work A/B samples	0.006	0.0	–	–	–
60	Finalize global fine publication	0.006	0.0	–	–	–
66	Fine JFA – reset recurring frontiers	0.006	0.0	–	–	–
37	Clear exact global fine page delta	0.006	0.0	–	–	–
33	Clear structured fine transport delta header	0.005	0.0	–	–	–
62	Fine-to-coarse restriction – restrict	0.001	0.0	17.5	32.2	–

S3a Advect the face velocities, apply forces, assemble the RHS

3.34 ms 1.9 % of frame 8 encoders t 56–60 ms structured dynamics

GOAL

Advect the normal velocity component stored on every power-diagram face; apply gravity and re-impose solid/free-slip constraints; assemble the divergence right-hand side $\rho \cdot \text{flux} / dt$ for every liquid row.

HOW THE 2017 PAPER IS REALISED

“we compute full velocity vectors at the centroid of each face and advect them using a standard semi-Lagrangian update, subsequently projecting the velocity onto the face normal direction.”

Aanjaneya et al. 2017, §5 – Velocity interpolation and advection

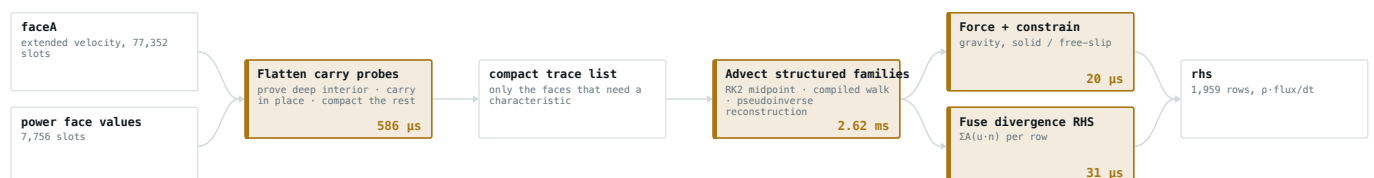
`advect()` does precisely that, with a second-order midpoint backtrace: sample at the face → sample at the half-step → depart from there → sample →

`canonicalVelocityDot(transported, n)`. The cell-centre full velocity vectors come from the catalog's stored pseudoinverse columns — the paper's "least squares fit based on all the face normal components" precomputed per topological case rather than solved at runtime.

§6.2 point location, compiled. The paper "retrieve[s] the local Delaunay tetrahedralization from the byte-encoded vertex table" and searches it. We now compile that same table once into an adjacency graph with per-octant entry points, so a query *walks* to its tetrahedron instead of scanning the fan. Nothing about the discretization changed — only how the lookup is spelled.

§4.1 divergence, eq (4). Fuse structured divergence RHS rows evaluates $V_{cell} \nabla \cdot u = \sum_{faces} A_{face} (u \cdot n)$ per row, reading face areas from the accepted `PowerRowMetric` bank. Zero dry-identity divergence RHS rows handles the disjoint dry-identity row class.

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE – WHAT THE GPU ACTUALLY DID

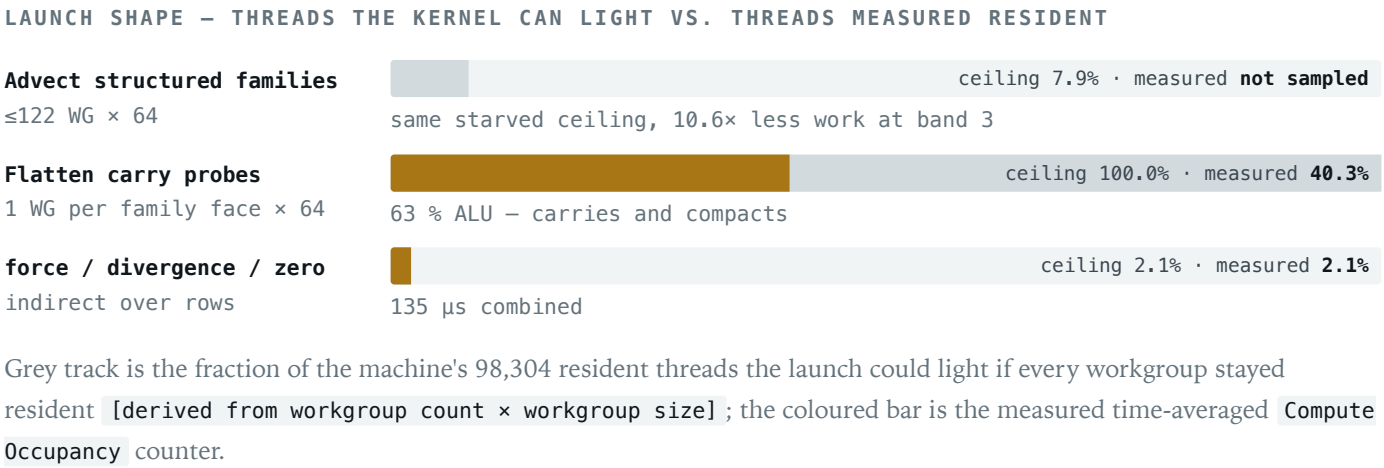
This band used to be the worst in the frame; it is now 1.9 % of it. Advection went 19.65 → 1.86 ms when the sampler was compiled, and its per-advance range collapsed

from 1.58 – 24.28 ms to 1.86 – 2.75 ms [both at band 3] . The range mattered more than the mean: this was the one encoder that made the whole frame wall unpredictable. At band 4 it reads 2.62 ms over 7,756 face slots, still flat.

Two independent fixes, and they compose. Flatten structured boundary carry probes (0.59 ms, 40 % occupancy, 63 % ALU — the band's only encoder still long enough for a counter sample to land in) now runs one workgroup per *family* face, proves the deep-interior carry predicate with a pair of eight-lane OR trees, *completes* the carry in place, and appends everything else to a dense list. Advect structured families then dispatches over that list only — so a SIMD group is uniformly traced, and the carried/traced divergence that produced the old bimodality cannot occur.

The second fix is inside the sampler: **one fan evaluation instead of up to eight**, a walk instead of a scan, and table lookups instead of two nested linear searches. The mechanism below unpacks all three, with the before/after code side by side.

Everything else in the band is still free. Forces, constraints, divergence and identity zeroing total 135 μ s for 1,959 rows and 7,756 faces — three storage reads and a fused multiply-add per face. The launch shape has *not* changed: still @workgroup_size(64) over ≤ 122 workgroups, still a 7.9 % residency ceiling. We did not widen it; we deleted the work. One consequence worth noting: at 2.62 ms the encoder is now *too short for the device-wide counters to sample at all* — its occupancy and ALU read — in the appendix, where they used to read 0.7 % and 1.9 %.



ACTUALLY HEALTHY dense arithmetic, or a launch whose work was deleted rather than widened

Was the worst band in the frame; is now 1.9 % of it. The launch shape did not change — still a 7.9 % ceiling — but the carry/trace divergence is gone (separate uniform passes) and the

sampler's three nested searches became table lookups. **20.14** → **2.40 ms at band 3**, 3.34 ms here.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked **[derived]**.

Point location: from a search inside a search, to a walk

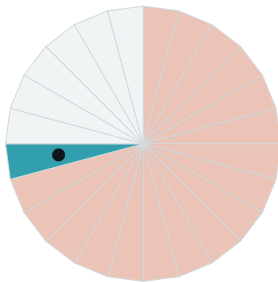
Advect structured families

Every power-diagram face carries one scalar — the normal component of velocity — and this kernel semi-Lagrangian-advects the ones that need it. It used to cost **19.65 ms and swing 15×**. It now costs **2.62 ms and swings 1.5×**, on the same launch shape, the same 7,756 faces, and the same second-order midpoint backtrace. Nothing was widened; the work was removed.

Three things were nested inside every characteristic sample, and each was a search where a lookup would do. The figure walks the innermost one — **which tetrahedron of the local Delaunay fan contains this point?** — and the code tabs show the other two.

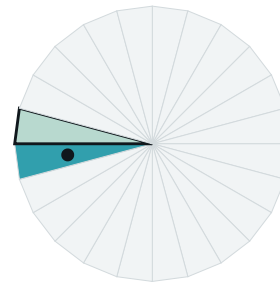
FINDING THE CONTAINING TETRAHEDRON — SOURCE-ORDER SCAN VS. OCTANT-SEEDED WALK

BEFORE — source-order scan



18 of 24 tested, from index 0 — $\times 4$ symmetries = 72

AFTER — octant-seeded walk



2 of 24 tested — one orientation, entered at the outlined wedge

selector-table lookups, the search inside the search: 8,208 comparisons → 3 indexed reads

fan size **24** tetrahedra candidates tested **72** → **2** selector comparisons **8,208** → **3**

encoder **19.65** → **2.62** ms per-advance range **15×** → **1.5×**

The fan is drawn as a ring of wedges, which is the 2D shadow of the real 3D star of tetrahedra around the cell anchor; the walk's step rule (cross the face whose barycentric weight is most negative) is the one in the shader. Candidate counts follow from the two strategies and the fan size [derived]; the encoder totals beneath are measured.

THE CODE THAT PRODUCES IT

1 · eight fans → one	2 · scan → adjacency walk	3 · the search inside the search	
4 · killing the divergence	the midpoint trace (unchanged)		
lib/webgpu-octree-structured-dynamics.ts · from line 1820		WGSL · reformatted whitespace only	

```
1 // BEFORE – evaluate every automorphism in the case's stabilizer and average.
2 //   for (var symmetry = 0u; symmetry < 8u; symmetry += 1u) {
3 //       let transform = d4[8u*fixedAxis + symmetry];
4 //       if ((symmetryMask & (1u << symmetry)) == 0u) { continue; }
5 //       let sample = transitionFanSample(row, local, first, count, transform);
6 //       xTerms[n] = sample.x; yTerms[n] = sample.y; zTerms[n] = sample.z; n += 1u;
7 //   }
8 //   return (1./f32(n)) * vec3f(canonicalInterpolation8(xTerms, n), ...);
9
10 // AFTER – pick the one canonical orientation from a compiled table.
11 let rowTransform      = metrics[row].transformAndFlags & 63u;
12 let canonicalVertical = abs(powerTransform(vec3f(0.,1.,0.), rowTransform));
13 let fixedAxis         = select(select(2u,1u,canonicalVertical.y>.5), 0u,
    canonicalVertical.x>.5);
14 let symmetryMask      = (catalog[thAt + 2u] >> (8u + 8u*fixedAxis)) & 255u;
15
16 let worldTransform    = canonicalWorldPointTransform(x);
17 let effectiveTransform = compiledTransformCompose(compiledInverseTransform(worldTransform),
    rowTransform);
18 let fanTransform      = compiledCanonicalFanTransform(fixedAxis, symmetryMask,
    effectiveTransform);
19 return transitionFanSample(row, metrics[row].caseId, local, first, count, fanTransform);
```

A generated case may admit several equivalent row transforms. The old sampler evaluated the whole stabilizer coset and averaged — up to **eight complete fan searches per interpolation**, done purely to make a reflected row land on the same answer as its mirror. The compiled table instead names the *minimum effective source orientation* in that coset, so both sides of a reflection select the same fan by construction. Same invariance, **one** evaluation. This is the largest of the three factors.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
Advect structured families		2.25	1.81	2.65	2.62

Advection reads **mean 2.25, min 1.81, max 2.65 ms** across the 42 advances in the window — a 1.5× spread, against the 15× it showed before the sampler was compiled. The old note called this "the least quotable number in the document"; the deep-interior carry it blamed is still the switch, but it is now resolved in a separate, uniform pass, so hitting or missing it no longer changes what a 32-lane group costs.

EVERY ENCODER IN S3A, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
71	Advect structured families	2.620	78.4	3.3	6.9	1.44
70	Flatten structured boundary carry probes	0.586	17.5	40.3	62.9	1.01
69	Prepare accepted structured dynamics worksets	0.041	1.2	53.9	74.1	1.00
75	Fuse structured divergence RHS rows	0.031	0.9	—	—	—
73	Prepare accepted structured dynamics worksets	0.022	0.7	53.9	74.1	1.00
74	Force and constrain structured families	0.019	0.6	—	—	—
72	Commit advected structured families	0.016	0.5	—	—	—
76	Zero dry-identity divergence RHS rows	0.006	0.2	—	—	—

S3b 1,959 unknowns, one workgroup, 256 threads

53.02 ms 30.7 % of frame 1 encoders t 60–113 ms pressure solve

GOAL

Solve $L_2p = f$ for 1,959 unknowns to a relative residual of 7.10×10^{-5} .

HOW THE 2017 PAPER IS REALISED

“(1) Starting with a zero initial guess $p_0 = 0$, execute k iterations of the damped Jacobi method on $L_2p = q$. This is only applied to a band (about 3 voxels wide) around the boundaries and level transitions. ... (2) Compute a correction $\delta p = M_1 r_1$ by applying the first order accurate preconditioner M_1 from Setaluri et al. ... (3) Repeat k additional iterations of the same Jacobi method ... It was necessary to perform an adequate number ($k \approx 8$) of boundary smoothing iterations.”

Aanjaneya et al. 2017, §4.3 – Multigrid preconditioner

`section43Correction` is exactly that three-phase structure, with the paper's constants: $k = 8$, band 3 layers, and an inner first-order V-cycle over the 6-level pyramid (pre-smooth + restrict down 5 levels, exact 1-cell bottom, prolong + post-smooth back up) with a Chebyshev smoother of degree 2 or 4. The outer loop is matrix-free CG on L_2 with an encoded budget of 10 and a hard ceiling of 16. **It converged in 5.**

§6.3 hierarchical operator. `applyAllRows` fetches 19 coefficients per row (centre + 6 face + 12 edge neighbours) from the catalog-resolved row template, with ghost contributions folded by the `GhostValuePropagate` / `GhostValueAccumulate` pair the paper describes.

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE – WHAT THE GPU ACTUALLY DID

The entire solve is one dispatch of one workgroup of 256 threads. `dispatchShape = [1,1,1]`, `@compute @workgroup_size(256)`.

Measured: **occupancy 0.2 %**, **ALU 0.2 %**, **placement 4.00×** — pinned to exactly one of the 4 GPU partitions, three at literally 0.00 %. 0.2 % of 98,304 threads is 197: the counter is reporting the 256 lanes plus quantisation noise. **The measurement is an exact confirmation of the structure — for 44 – 61 ms of every 180 ms frame, 99.8 % of this GPU is switched off.**

Why it is written this way is legible and not an accident. The kernel is *persistent*: state lives in a workgroup-private arena of 11 channels (x, r, z, d, A·d, four compensated-f32 fields, two band u32 fields), every phase is separated by `storageBarrier()`; `workgroupBarrier()`; , and the compensated summation tree emulates 128-lane virtual workgroups so the association order of every f32 addition is fixed. That is what makes the solve **bitwise reproducible** — precisely what the D4 symmetry gate on this scene requires. A multi-workgroup solve needs a device-wide barrier, which WGSL does not have.

The cost is latency, not throughput. Counting barrier-separated phases from the code [derived] : per CG iteration ≈ 1 (x/r update) + ~ 60 (`section43Correction` : 7 pre-sweeps $\times 2$, an `applyAllRows` , the inner RHS, ~ 25 in `vcycleCorrection` across 6 levels, 8 post-sweeps $\times 2$) + ~ 4 (merged reduction) + 1 (β) + 1 (apply) + ~ 2 (curvature) ≈ 68 ; over 5 iterations plus setup, ≈ 355 **barrier-separated phases**. $52.3 \text{ ms} / 355 \approx 147 \mu\text{s}$ **per phase**. Each phase is one strided pass over $\leq 1,959$ rows by 256 lanes — 8 rows per lane, each a 19-entry gather — so ~ 155 dependent loads per lane with **8 resident SIMD groups and therefore essentially no latency hiding**.

In absolute terms: **1,959 unknowns per 52.3 ms is 37,493 unknowns per second**. The gate that selects this executor is `OCTREE_PERSISTENT_MGPCG_MAXIMUM_ROW_CAPACITY = 65,536` — every scene at or below 65k pressure rows takes this path.

LAUNCH SHAPE – THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT

persistentMGPCG

1 WG $\times$ 256

three of 4 partitions at 0.00 %

ceiling 0.3% · measured **0.2%**

Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident [derived from `workgroup count  $\times$  workgroup size`] ; the coloured bar is the measured time-averaged `Compute Occupancy` counter.

THREAD-STARVED BY CONSTRUCTION the launch cannot fill the machine before it executes an instruction

One workgroup. 256 threads of 98,304. Occupancy 0.2 %, placement 4.00×, three GPU partitions at literally 0.00 % for 44 – 61 ms. Deliberate — bitwise determinism — but a trade, and it constrains the reductions, not the applies. Now **31 % of the frame**.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked **[derived]**.

A whole Poisson solve inside one workgroup, serialised by barriers

Octree persistent MGPCG – whole solve in one workgroup

The executor is **one dispatch of one workgroup of 256 threads**. Every quantity the CG iteration needs lives in a workgroup-private arena; every phase is separated by `storageBarrier(); workgroupBarrier();`, because that pair is the only synchronisation WGSL offers and a multi-workgroup solve would need a device-wide barrier that does not exist. The consequence is that the solve's cost is $\text{latency} \times \text{phase count}$, not throughput: each phase is one strided pass over $\leq 1,959$ rows by 256 lanes, so 8 rows per lane, each row a 19-entry gather. With 8 resident SIMD groups there is essentially nothing to hide that latency behind.

Step the phase ladder below to see what one CG iteration actually costs. The reason it is written this way is the second tab: `canonical18Sum` sorts the stencil's pair sums before adding them, which is what makes the result invariant under the 48 signed axis permutations — and that invariance is exactly what the D4 symmetry gate on this scene tests.

ONE CG ITERATION, PHASE BY PHASE – 5 ITERATIONS PLUS SETUP ≈ 355 PHASES

1 / 67 · x / r update



single phase

256 resident threads of 98,304 – for every one of these phases

phase 1 of 67 $\approx 123 \mu\text{s}$ each [derived] elapsed in this iteration 0.12 ms rows per lane 8.2

machine idle 99.7 %

Phase count is counted from the source [derived]; $123 \mu\text{s}/\text{phase}$ is $43.51 \text{ ms} \div 355$.

Group boundaries are exact — every one is a `storageBarrier(); workgroupBarrier();`

pair, the only device-visible synchronisation WGSL offers inside a dispatch.

THE CODE THAT PRODUCES IT

the dispatch

the CG loop

the preconditioner

why it must be serial

the \$6.3 operator

lib/webgpu-octree-persistent-mgpcg.ts · from line 632

TS · ... marks an elision

```
1 readonly dispatchShape = [1, 1, 1] as const;
2 ...
3 pass.dispatchWorkgroups(...this.dispatchShape); // line 1081
```

One workgroup. The kernel is declared `@compute @workgroup_size(256)` (`lib/webgpu-octree-persistent-mgpcg.wgsl.ts:1596`). 256 of 98,304 threads is 0.26 % of the machine's residency, and the counter measures 0.3 % — the structure and the measurement agree exactly.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
Octree persistent MGPCG – whole solve in one workgroup		52.25	44.20	61.27	53.02

The solve swings 44.20 → 61.27 ms — CG converges in 4 – 6 iterations depending on the step. This capture's representative advance is near the top of that range, the previous one was at the bottom, so **compare means, not representatives**: 56.34 ms then, 52.25 ms now — unchanged, as expected, since neither shipped optimisation touches this kernel. It is now the only task on the page that is not flat to ±5 %.

HEADROOM IN THIS BAND

1. **Widen the pressure solve** ≈52 ms — The largest item, and the only one still swinging. The determinism requirement is real, but it constrains the *reduction* order, not the stencil applies. A hybrid — wide multi-workgroup applies and smooths, with the compensated reductions kept in a single deterministic workgroup — keeps exactness exactly where the D4 gate needs it.

EVERY ENCODER IN S3B, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
77	Octree persistent MGPCG – whole solve in one workgroup	53.020	100.0	0.2	0.2	4.00

S3c Apply the gradient – 116 microseconds

0.12 ms 0.1 % of frame 4 encoders t 113–113 ms projection

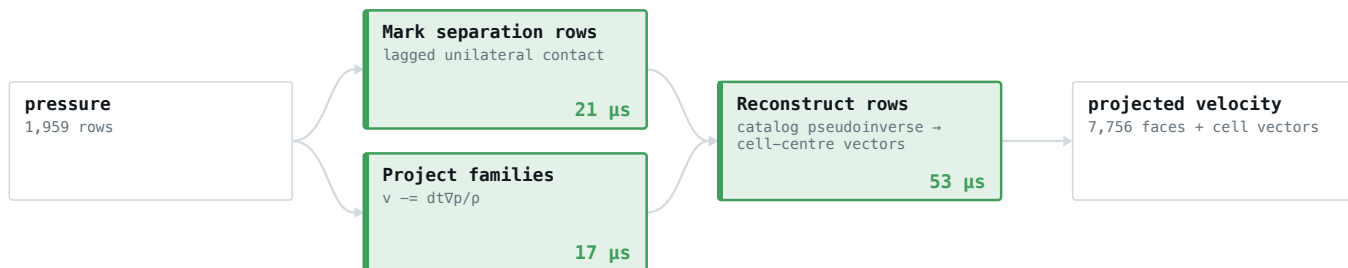
GOAL

Apply $\mathbf{v} \leftarrow \mathbf{v} - \mathbf{dt} \cdot \nabla p / \rho$ on every power face, mark the lagged unilateral-contact separation set from the solved pressure, and reconstruct cell-centre velocity vectors for the next advance's samplers.

HOW THE 2017 PAPER IS REALISED

§4.1 discrete gradient. Because the power diagram restores primal-dual orthogonality, the pressure gradient across a face is a single difference along the dual edge, scaled by the stored inverse distance. **No T-junction special case exists** — that is the entire point of the power-diagram construction. Reconstruct projected structured rows then applies the catalog's stored pseudoinverse to rebuild full vectors at cell centres.

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE – WHAT THE GPU ACTUALLY DID

116 μs for the entire projection. Project families 17 μs · reconstruct rows 53 μs · prepare and separation marking the rest.

This is why the paper's "Projection" line is 30 – 49 % while ours is dominated by the *solve*: applying the gradient is 17 μs; solving for p is 53,020 μs. The ratio is 3,119 : 1.

LAUNCH SHAPE – THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT

project families

≤122 WG × 64

17 μs – no divergence, no trace

ceiling 7.9% · measured 7.9%

Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident [derived from workgroup count × workgroup size]; the coloured bar is the measured time-averaged Compute Occupancy counter.

ACTUALLY HEALTHY dense arithmetic, or a launch whose work was deleted rather than widened

116 μ s, nothing to report. Applying the gradient is 17 μ s against the solve's 53,020 — a ratio of 3,119 : 1. The paper's expensive "projection" line is, here, almost entirely the solve.

EVERY ENCODER IN S3C, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
81	Reconstruct projected structured rows	0.053	45.5	—	—	—
79	Mark structured overhead separation rows	0.023	19.9	—	—	—
78	Prepare accepted structured dynamics worksets	0.023	19.9	53.9	74.1	1.00
80	Project structured families	0.017	14.7	—	—	—

S3d Redistance, correct volume, restrict to the octree

11.08 ms 6.4 % of frame 76 encoders t 113–127 ms fine settlement

GOAL

The transported generation was published before the projection so coarse ϕ could be corrected before forces. Now that the projected velocity and its closest-point extension exist, settle it: redistance the narrow band, correct volume, restrict the fine level set onto the coarse octree, refresh the summary mips.

HOW THE 2017 PAPER IS REALISED

“We are presently reinitializing the signed distance with a serial Fast Marching method on the narrow band; alternative reinitialization schemes that admit parallelism certainly merit attention in future work.”

Aanjaneya et al. 2017, §8 – Limitations and future work

We took that future work. A jump-flooding closest-point transform: seed the interface, flood $A \leftrightarrow B$ down the descending ladder $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ and then run **two stride-1 collar repairs**, and resolve distance from the winning closest point. The ladder is sized so its *summed* reach $2P - 1 = 15$ covers the band, not so its first stride does. Warm generations reseed only dirty pages via a staged frontier; if the on-GPU resolve records show a collar miss, a fully on-GPU fallback replays the complete cold ladder — the same $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ descent followed by all **five** collar repairs.

Volume correction — not in the paper. Three reduction/correction rounds hold `volumeDrift` at -3.77×10^{-6} over 299 steps. That is well above the pre-packing -1.14×10^{-7} , and the cause is legible: ϕ is now stored as binary16 (see S2). Still six orders below the cell volume, and still a guarantee the paper does not make.

“Subsequently, we use the fine level set to correct the coarse level set wherever we have valid ϕ -values.”

Aanjaneya et al. 2017, §5

Fine-to-coarse restriction · prepare / restrict / publish then Power coarse level set · persistent schedule.

DATAFLOW — BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE — WHAT THE GPU ACTUALLY DID

The JFA kernels are the healthiest code in the frame. 51 % occupancy, 73 % ALU. One voxel per lane over the resident support pages; each stride pass a fixed 27-neighbour gather with no divergence. This is what the rest of the frame should look like.

The conditional fallback ladder is the number to distrust in this band. It costs 0.01 ms here and 8.28 ms on the capture two back — same guard, same scene, opposite predicate outcomes. The warm path is stable at 4.07 ms, so the honest statement is that the band costs **11.1 ms or 19.4 ms depending on a predicate** nothing has made more predictable.

The warm ladder's shape is worth seeing: strides 8 and 4 cost **~1.4 μs each** because the warm frontier is empty at that reach, while each of the four stride-1/2 passes costs ~0.89 ms. Nearly all the cost is the tail, not the reach.

The three volume rounds (4.32 ms) are pure reductions at 16.6 % occupancy but 63 – 66 % ALU — small, arithmetic-dense, and the price of a volume-drift guarantee the paper does not make.

LAUNCH SHAPE — THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT

JFA flood (stride pass) per support page × 256	ceiling 100.0% · measured 51.0%
	73 % ALU — the healthiest kernel here
JFA fallback ladder full support	ceiling 100.0% · measured 0.0%
	did not fire this advance; 8.28 ms when it does
volume reductions partial-sum tree	ceiling 100.0% · measured 16.6%
	63–66 % ALU

Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident [derived from workgroup count × workgroup size]; the coloured bar is the measured time-averaged **Compute Occupancy** counter.

ACTUALLY HEALTHY dense arithmetic, or a launch whose work was deleted rather than widened

The healthiest code in the frame — 51 % occupancy, 73 % ALU, one voxel per lane, no divergence. The band's cost is not the ladder but the *conditional* fallback replay, which is **0.01**

ms here and 8.28 ms two captures back. Same predicate, two answers.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked **[derived]** .

The one place we beat the paper, and the guard whose cost is a coin flip

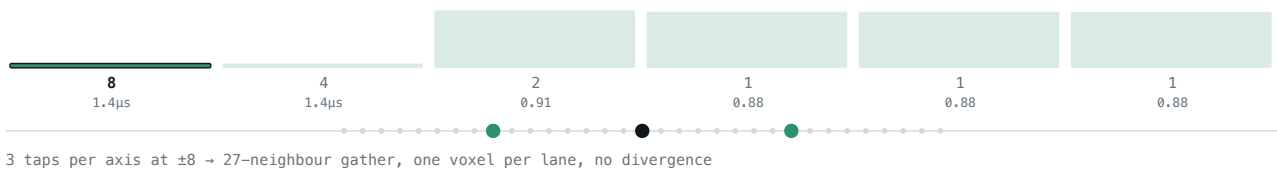
Fine JFA – cooperative flood

§8 of the paper names its serial fast-marching reinitialisation as the thing that most wants parallelising. We replaced it with a jump-flooding closest-point transform, and it is the healthiest code in the frame: 53 % occupancy, 73 % ALU, one voxel per lane, a fixed 27-neighbour gather per pass with no divergence. Reinitialisation is 2.2 % of our frame against 23 – 26 % of the paper's.

The interesting part is the ladder's *sizing*, and the fallback. Step the passes below: the warm ladder's first two strides cost $1.4\ \mu\text{s}$ because the frontier is empty at that reach. The conditional cold replay is idle on this advance and cost **8.28 ms** two captures back — same guard, same scene, different predicate outcome. Toggle it to see both frames.

THE DESCENDING LADDER, AND THE FALLBACK THAT REPLAYS IT

1. cooperative flood A-B stride 8



pass 1 of 6 stride 8 cumulative reach 8 cells cumulative 1.4 µs of 3.56 ms

occupancy 53 % · ALU 73 %

Every per-pass millisecond here is measured from the representative advance. Reach is the running sum of strides — the sizing rule the source calls out: a schedule starting at P reaches $2P - 1$.

THE CODE THAT PRODUCES IT

sizing the ladder	why not a sweep
lib/webgpu-octree-fine-levelset-redistance.ts · from line 127	
TS	
<pre> 1 export const FINE_LEVELSET_JFA_COLD_COLLAR_REPAIR_PASSES = 5; 2 export const FINE_LEVELSET_JFA_WARM_COLLAR_REPAIR_PASSES = 2; 3 4 export function planFineLevelSetJFAStrides(5 bandCells: number, 6 maximumDisplacementFineCells = 4, 7 collarRepairPasses = FINE_LEVELSET_JFA_COLD_COLLAR_REPAIR_PASSES, 8): readonly number[] { 9 let stride = 1; 10 const repairRadius = Math.min(bandCells, maximumDisplacementFineCells); 11 if (maximumDisplacementFineCells >= bandCells) { 12 // Reach is the SUM of the descending strides, not the first stride: a 13 // schedule starting at P sums to P + P/2 + ... + 1 = 2P - 1 ... 14 while (stride * 2 - 1 < repairRadius) stride *= 2; 15 } else { 16 while (stride * 2 <= repairRadius) stride *= 2; 17 } 18 const strides: number[] = []; 19 for (; stride >= 1; stride /= 2) strides.push(stride); 20 // Five +1 collar repairs close sparse page-boundary landing gaps ... a 21 // closest-point route may turn across a page corner and therefore need 22 // several local hops after the binary landing ladder. 23 for (let repair = 0; repair < collarRepairPasses; repair += 1) strides.push(1); 24 return strides; 25 }</pre>	

The $2P - 1$ observation is worth a pass on its own: choosing the smallest P whose *summed* reach covers the band, rather than rounding the band up to a power of two, removes one full flood. Band 23 covers via $16 + 8 + 4 + 2 + 1 = 31$; the naive rule picks 32 and spends its first pass gathering entirely outside the band.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
Fine JFA – cooperative flood B to A stride 1		2.04	2.00	2.17	2.02
Reduce resident fine overlap partials		1.38	1.34	1.47	1.36
Re-reduce corrected global fine volume		1.38	1.34	1.48	1.42
Measure twice-corrected global fine volume		1.37	1.34	1.46	1.35
Fine JFA – cooperative flood A to B stride 2		1.04	1.03	1.14	1.04
Fine JFA – cooperative flood A to B stride 1		1.01	1.00	1.10	1.01

3. Make the JFA fallback ladder conditional in fact, not just in form `≈8 ms` — `≈8 ms` on the advances where it fires and provably changes nothing —

`redistanceUnresolvedCells: 0` and `redistanceFallbackPages: 0` in the end-of-run audit. Two captures ago it fired and cost that; the last two it did not. Same predicate, and nothing has made it more predictable.

EVERY ENCODER IN S3D, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
123	Re-reduce corrected global fine volume	1.420	12.8	11.1	43.7	1.33
120	Reduce resident fine overlap partials	1.360	12.3	16.6	65.8	1.00
126	Measure twice-corrected global fine volume	1.350	12.2	16.7	64.9	1.00
95	Fine JFA – cooperative flood A to B stride 2	1.040	9.4	43.8	72.3	1.11
97	Fine JFA – cooperative flood B to A stride 1	1.010	9.1	53.3	72.9	1.00
98	Fine JFA – cooperative flood A to B stride 1	1.010	9.1	53.4	72.9	1.01
99	Fine JFA – cooperative flood B to A stride 1	1.010	9.1	53.3	72.9	1.00
92	Fine JFA – seed closest points	0.458	4.1	16.6	17.2	–
135	Power coarse level set – persistent schedule	0.376	3.4	0.4	0.9	2.53
102	Fine JFA – resolve A to B	0.270	2.4	–	–	–
157	Publish direct fine-summary directory and active mip	0.269	2.4	0	0	1.00
132	Fine-to-coarse restriction – restrict	0.269	2.4	17.5	32.2	–
134	Power coarse level set – publish exact dispatch	0.161	1.5	–	–	–
150	Recompute direct fine-summary bases	0.148	1.3	–	–	–
143	Ensure sparse fine-summary directory pages	0.092	0.8	–	–	–
156	Recompute direct fine-summary parents level 5	0.048	0.4	–	–	–
121	Finalize global fine volume	0.044	0.4	–	–	–
155	Recompute direct fine-summary parents level 4	0.043	0.4	–	–	–
152	Recompute direct fine-summary parents level 1	0.043	0.4	–	–	–
145	Ensure compact fine-summary ranks	0.041	0.4	–	–	–
124	Finalize first corrected global fine volume	0.040	0.4	–	–	–
127	Finalize measured global fine volume	0.040	0.4	0	0	1.00
154	Recompute direct fine-summary parents level 3	0.039	0.4	–	–	–
153	Recompute direct fine-summary parents level 2	0.036	0.3	–	–	–
105	Fine JFA – prepare warm fallback	0.032	0.3	–	–	–
118	Fine JFA – finalize	0.029	0.3	–	–	–
125	Apply residual bounded global fine normal correction	0.027	0.2	–	–	–
133	Fine-to-coarse restriction – publish	0.027	0.2	–	–	–
119	Fine JFA – commit distances	0.026	0.2	–	–	–
122	Apply bounded global fine normal correction	0.026	0.2	–	–	–
140	Remove direct fine-summary page references	0.022	0.2	–	–	–
129	Settle deferred global fine publication	0.022	0.2	–	–	–
130	Settle deferred rejected fine work payload	0.015	0.1	–	–	–
103	Finalize compact coarse volume	0.015	0.1	–	–	–
147	Add direct fine-summary page references	0.015	0.1	–	–	–
104	Prepare active global fine volume dispatch	0.014	0.1	–	–	–
139	Retire direct coarse summary ranks	0.014	0.1	0	0	1.00
144	Ensure sparse coarse-summary directory pages	0.013	0.1	–	–	–
136	Prepare direct fine-summary publication	0.013	0.1	–	–	–
148	Publish direct coarse summary ranks	0.012	0.1	0	0	1.00
146	Ensure compact coarse-summary ranks	0.012	0.1	–	–	–

#	ENCODER	MS	%	OCC	ALU	PLC
101	Reduce compact coarse volume partials	0.011	0.1	—	—	—
138	Validate fine-summary coarse rows	0.010	0.1	—	—	—
96	Reset global volume reduction	0.010	0.1	—	—	—
131	Fine-to-coarse restriction – prepare	0.009	0.1	—	—	—
128	Finalize global fine publication	0.009	0.1	—	—	—
142	Reclaim empty fine-summary directory pages	0.009	0.1	—	—	—
100	Prepare exact compact coarse volume dispatch	0.009	0.1	—	—	—
137	Validate exact fine-summary delta	0.007	0.1	—	—	—
151	Stamp octree topology attempt generation	0.006	0.1	—	—	—
149	Prepare active fine-summary mip dispatch	0.006	0.1	—	—	—
141	Prepare sparse fine-summary directory-page reclamation	0.005	0.0	—	—	—
91	Fine JFA – finalize recurring frontier dispatches	0.005	0.0	—	—	—
82	Fine JFA – clear recurring frontier masks	0.002	0.0	—	—	—
86	Fine JFA – build recurring frontier 4	0.002	0.0	—	—	—
83	Fine JFA – mark recurring dirty frontier	0.002	0.0	—	—	—
84	Fine JFA – build recurring frontier 6	0.002	0.0	—	—	—
85	Fine JFA – build recurring frontier 5	0.002	0.0	—	—	—
88	Fine JFA – build recurring frontier 2	0.002	0.0	—	—	—
89	Fine JFA – build recurring frontier 1	0.002	0.0	—	—	—
90	Fine JFA – build recurring frontier 0	0.002	0.0	—	—	—
93	Fine JFA – cooperative flood A to B stride 8	0.002	0.0	—	—	—
87	Fine JFA – build recurring frontier 3	0.002	0.0	—	—	—
94	Fine JFA – cooperative flood B to A stride 4	0.002	0.0	—	—	—
107	Fine JFA – seed full-support fallback closest points	0.002	0.0	—	—	—
109	Fine JFA – complete support fallback 2 stride 4	0.002	0.0	—	—	—
111	Fine JFA – complete support fallback 4 stride 1	0.002	0.0	—	—	—
112	Fine JFA – complete support fallback 5 stride 1	0.002	0.0	—	—	—
113	Fine JFA – complete support fallback 6 stride 1	0.002	0.0	—	—	—
114	Fine JFA – complete support fallback collar repair 3	0.002	0.0	—	—	—
115	Fine JFA – complete support fallback collar repair 4	0.002	0.0	—	—	—
116	Fine JFA – complete support fallback collar repair 5	0.002	0.0	—	—	—
117	Fine JFA – resolve complete support fallback	0.002	0.0	—	—	—
106	Fine JFA – refresh full-support fallback closest-point codes	0.001	0.0	—	—	—
108	Fine JFA – complete support fallback 1 stride 8	0.001	0.0	—	—	—
110	Fine JFA – complete support fallback 3 stride 2	0.001	0.0	—	—	—

S3e The same 34 ms extrapolation, a second time

37.31 ms 21.6 % of frame 10 encoders t 127–164 ms Sec 5 support (settled)

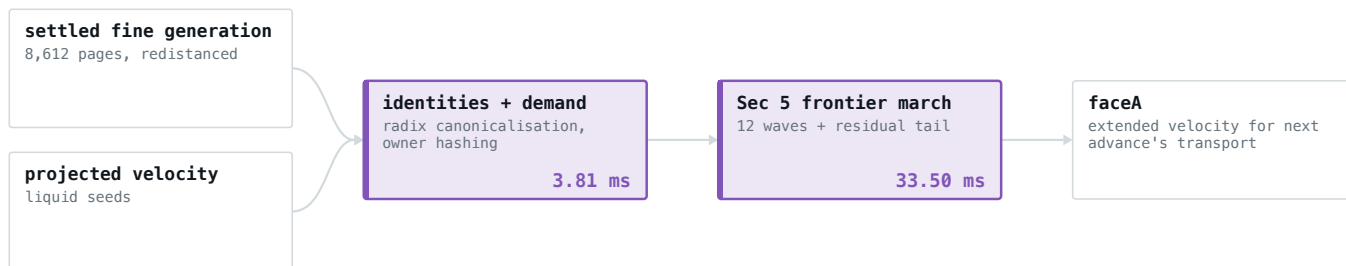
GOAL

Byte-for-byte the same producer as S1 passes 10 – 19, invoked with `site = "settled-fine"` instead of `"topology-commit"`. S1's rebuild targets the newly accepted epoch; this one targets the fine generation that just settled, so next advance's transport has an extended velocity field consistent with the surface it will sample.

HOW THE 2017 PAPER IS REALISED

Identical to S1: \$5 velocity extrapolation on faces, seeded from the liquid and marched outward to a fixed point, then reconstructed back onto power faces.

DATAFLOW – BUFFERS, KERNELS, MEASURED TIME



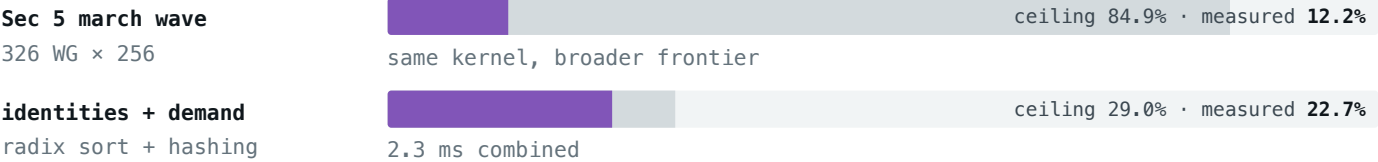
PERFORMANCE – WHAT THE GPU ACTUALLY DID

Two full frontier marches per advance is 65.61 ms — 38 % of the frame, and now the frame's largest line item after the solve. Both solve the same 77,352-unknown extrapolation problem over graphs that differ only where the topology or the surface moved. Nothing in the last two commits touched this; it simply became the biggest thing left.

The producer already has a changed-frontier mode that seeds only altered faces and carries settled ones. The measurement says that on this scene the "changed" frontier is still most of the domain, or that the two sites cannot share carry state. Either way it is an implementation question, not a numerical one — the paper needs the extended field to exist once per surface update.

This site measures slightly higher occupancy than S1 (12.2 % vs 9.9 %) and higher ALU (26.0 % vs 22.3 %), consistent with a broader live frontier after the surface moved — on the same fixed 326-workgroup grid.

LAUNCH SHAPE — THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT



Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident `[derived from workgroup count × workgroup size]` ; the coloured bar is the measured time-averaged `Compute Occupancy` counter.

LAUNCHED WIDE, DRAINED NARROW a provisioned-capacity grid over a work set that is orders smaller
Byte-for-byte the same producer as S1, on the same fixed 326-workgroup grid, for **33.50 ms**. A slightly broader live frontier lifts occupancy to 12.2 % — still a fifth of what the launch asked for.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked `[derived]` .

A sparse frontier dispatched on a dense, fixed grid

March Section 5 sparse changed frontier to a fixed point

Velocity has to exist everywhere the level set might backtrace into — including inside air — so §5 extends the liquid face velocities outward. The paper fast-marches this serially on the CPU. We relax a monotone frontier on the GPU: a queue of faces that *changed* in the previous wave expands through the reciprocal incidence graph, each destination re-runs a closest-face gather, and a face that moves re-enqueues itself. Twelve host-encoded waves, then a 3-workgroup residual tail that loops to a fixed point in-kernel.

The algorithm is sparse. **The launch is not.** The per-wave indirect record is only re-authored on the factor-1 path; here every wave dispatches one fixed record sized to the whole published face space. Drag the frontier size below and watch what that costs.

EVERY WAVE DISPATCHES 326 WORKGROUPS OVER ALL 77,352 FACE SLOTS

0 1 2 3 4 5 6 7 8 9 10 11 T prefix wave → expand → relax → commit → advance, 4 dispatches, one compute pass
83,456 lanes launched · 4,000 do work · 79,456 return on their first branch



```
if (local >= current) { return; }  
- expandAirSupportChangedFrontier, and the same guard in relax and commit  
residency this dispatch actually asks for: 4.07 % of the machine
```

dispatch 326 WG × 256 useful lanes 4.79 %

measured time-averaged occupancy 6.7 % (S1) / 11.1 % (S3e) 48 dispatch boundaries in one encoder

The 326-workgroup grid, the 77,352 slots and the 12 waves are measured. The frontier size is yours to set — per-wave frontier counts were not instrumented in this capture. Whatever you choose, the launch does not change: that is the point.

THE CODE THAT PRODUCES IT

the 12 waves expand relax + commit the gather (measured)

lib/webgpu-octree-air-velocity-support-gpu.ts · from line 1392 TS · ... marks an elision

```
1  const frontierWaves = octreeAirSupportParallelFrontierWaves(); // = 12
2  for (let wave = 0; wave < frontierWaves; wave += 1) {
3      const waveOffset = indirectFrontierGate ? 0 : 48; // gate is false at
      fineFactor 4
4      pass.setPipeline(this.pipelines.expandAirSupportChangedFrontier!);
5      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
6      pass.setPipeline(this.pipelines.relaxAirSupportChangedFrontier!);
7      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
8      pass.setPipeline(this.pipelines.commitAirSupportChangedFrontier!);
9      pass.dispatchWorkgroupsIndirect(indirectFor(waveOffset), waveOffset);
10     pass.setPipeline(this.pipelines.advanceAirSupportChangedFrontier!);
11     ...
12     pass.dispatchWorkgroups(1); // singleton: rotate the
      queues
13 }
14 // residual tail: exact, unbounded, one workgroup per velocity axis
15 pass.setPipeline(this.pipelines.marchAirSupportFacesChangedFrontier!);
16 pass.dispatchWorkgroups(3);
```

Record 48 is the whole face space. It is authored once, as `dispatchFor(s(29u), 256u)` where `s(29)` is the published face count `77,352 → 326 workgroups`. Wave 11 launches exactly as wide as wave 0. All 48 dispatches sit inside a single compute pass, which is why the profiler reports them as one 32.1 ms encoder — and why there are 48 device-wide serialisation points inside it.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
March Section 5 sparse changed frontier to a fixed point – settled-fine		32.88	30.77	35.54	33.50

HEADROOM IN THIS BAND

2. Halve the Sec 5 marches `≈33 ms` shared with S1 — Now unambiguously the biggest structural redundancy left: the same extrapolation, twice per advance. Share the extended field between the `topology-commit` and `settled-fine` sites, or make the changed-frontier seeding actually sparse on this scene. The paper needs the extended field to exist once per surface update; we build it twice.
5. Fix the wide-launch / narrow-drain shape of the march waves shared with S1 — Part of the same 33 ms. 12 host-encoded barrier waves over a provisioned-capacity grid, when the live frontier is a few thousand faces, is the textbook case for a persistent multi-workgroup kernel with an on-GPU queue. The source already measured that the gather inside it is only `~10 %` of the pass.

EVERY ENCODER IN S3E, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
166	March Section 5 sparse changed frontier to a fixed point – settled-fine	33.500	89.8	12.2	26	1.28
159	Publish structured air-support identities – settled-fine	0.991	2.7	28.8	12.1	–
167	Reconstruct Section 5 air-support vectors – settled-fine	0.919	2.5	4.5	10	1.73
160	Close sparse fine air-support demand – settled-fine	0.891	2.4	3.5	4.1	1.27
165	Extrapolate structured ordinary faces and reconstruct support vectors – settled-fine	0.484	1.3	6.6	7.9	–
162	Canonicalize footprint air-support identities – settled-fine	0.262	0.7	–	–	–
164	Resolve and hash live adaptive air owners – settled-fine	0.168	0.5	–	–	–
161	Emit sparse fine air-support demand – settled-fine	0.039	0.1	–	–	–
163	Publish canonical air-support candidates – settled-fine	0.029	0.1	–	–	–
158	Initialize structured air-support publication – settled-fine	0.026	0.1	–	–	–

S4 Build and validate the whole next epoch, inactive

13.15 ms 7.6 % of frame 68 encoders t 164–180 ms next-epoch candidate

GOAL

From this advance's surface and projected velocity, build a complete, validated, *inactive* candidate: refined octree, graded, with owner pages, a compact liquid frontier, power descriptors and catalog-resolved rows, structured velocity and boundary banks, and the SPGrid hierarchy — then cross-validate the whole thing in one reduction. S1 of the next advance either commits it atomically or discards it.

HOW THE 2017 PAPER IS REALISED

§6.1 topology encoding. Resolve structurally dirty power descriptors builds the 18-bit / 9-bit descriptors; Resolve affected power topology rows looks each up in the 8,083-configuration catalog and writes 19 stencil coefficients, face areas, normals and inverse distances into the candidate PowerRowMetric bank. This is the paper's "retrieve stencil coefficients via a look-up table, indexed by a compact descriptor of the local topology".

Grading. Octree resident grading closure enforces §6.1's restriction that a cell and all its neighbours span only two levels — the invariant that makes the descriptor encoding well-defined at all.

§4.2 pyramid. The 21-phase SPGrid V-cycle · candidate * chain builds each level's active set, detects and inserts ghost cells with the paper's power-diagram amendment (edge neighbours that share a power face count as neighbours when deciding whether to spawn a ghost), counts/scans/writes the inter-level transfers, links parent chains, builds stencils and publishes spectral bounds for the Chebyshev smoother.

§4.1 boundary conditions. Resolve canonical free-surface boundary slots (ghost-fluid θ) and Resolve canonical solid boundary slots (variational cut-cell apertures) — the paper's Brochu/Batty embedded-boundary treatment, needing only liquid signed distance at cell centres and solid signed distance at cell vertices.

DATAFLOW — BUFFERS, KERNELS, MEASURED TIME



PERFORMANCE — WHAT THE GPU ACTUALLY DID

The catalog lookup is nearly free — 200 μ s to resolve 1,956 descriptors and write their metrics. §6.1's central claim, that a lookup table beats explicit meshing, is comprehensively vindicated: the topology encoding costs 0.09 % of the frame.

The expensive parts are the delta machinery, not the physics. Compare topology-tile refinement signatures (1.90 ms), Octree resident grading closure (2.63 ms), SPGrid V-cycle · candidate build level sets (2.06 ms) and candidate link parent chains (1.30 ms) are all launched over the *provisioned* tile/level domain rather than the changed set, and all measure 0.1 – 1.0 % occupancy with placement 2.0 – 4.0 $\times$ — one to two partitions of four doing anything at all. Together they are 7.9 ms of the 13.0 ms band.

The whole candidate rebuild runs every advance (`quadtreeRebuildCadenceSteps: 1` , 240 rebuilds in 240 steps). The paper adapts topology per frame, not per substep, and books 5 s of 58 s (8.6 %) for it on the rotating-paddle scene. Ours is 5.8 % plus the 60 μ s commit — comparable, and cheap for what it buys: a fully validated, atomically-committed epoch with a rejection path.

LAUNCH SHAPE — THREADS THE KERNEL CAN LIGHT VS. THREADS MEASURED RESIDENT

tile signature compare		ceiling 100.0% · measured 0.1%
provisioned tile domain	placement 4.00 $\times$ – one partition	
grading closure		ceiling 100.0% · measured 1.0%
provisioned tile domain	placement 2.00 $\times$	
SPGrid candidate build		ceiling 100.0% · measured 0.7%
provisioned level domain	placement 4.00 $\times$	
descriptor + catalog		ceiling 2.1% · measured 2.1%
affected rows only	200 μ s for 2,100 rows	

Grey track is the fraction of the machine's 98,304 resident threads the launch could light if every workgroup stayed resident [derived from `workgroup count × workgroup size`]; the coloured bar is the measured time-averaged `Compute Occupancy` counter.

LAUNCHED WIDE, DRAINED NARROW a provisioned-capacity grid over a work set that is orders smaller

The physics is free: 1,956 descriptors resolved and their §6.3 metrics written in **200 μ s**. Four delta kernels sized by the *provisioned* tile/level domain are **7.84 ms of the band's 13.15**, at 0.1 – 1.0 % occupancy and placement 2.0 – 4.0 $\times$.

MECHANISM

What the GPU is actually doing in this band's expensive encoders, and the source that produces it. Diagrams are interactive; every number in them is either measured or marked **[derived]**.

Four kernels that walk the provisioned domain to find a changed set of hundreds

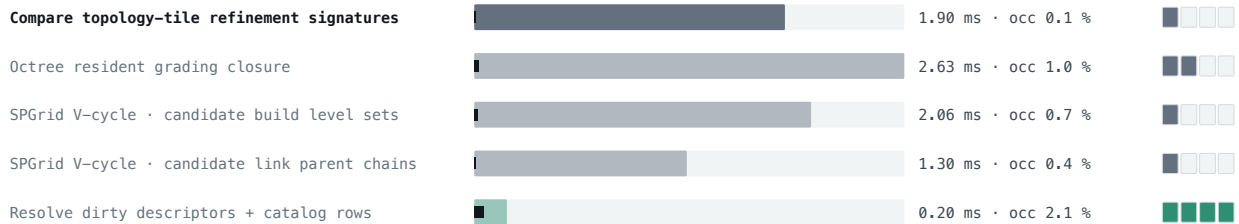
Compare topology-tile refinement signatures · grading closure · SPGrid candidate build

S4 builds the whole next epoch — refined octree, descriptors, catalog rows, structured and boundary banks, the SPGrid pyramid — and validates it, for 13.15 ms. The physics inside it is nearly free: resolving 1,956 descriptors and writing their \$6.3 metrics costs **200 μ s**, which is \$6.1's central claim vindicated at 0.12 % of the frame.

What costs is the delta machinery. Four kernels — tile-signature compare, grading closure, SPGrid level-set build, parent-chain link — are 7.84 ms of the 13.15, and every one of them is dispatched over the *provisioned* tile or level domain rather than the changed set. Their measured placement is 2.0 – 4.0×: one or two GPU partitions of four doing anything at all. Unchanged by the last two commits, and now **3.7 % of the frame** rather than 3.5 %.

FOUR KERNELS SIZED BY CAPACITY, ONE SIZED BY WORK

bar = GPU ms · inner mark = measured compute occupancy · squares = the four GPU partitions



Compare topology-tile refinement signatures 1.90 ms occupancy **0.1 %** placement **4.00×**

sized by the provisioned domain

Milliseconds, occupancy and placement are all measured per encoder. Placement is peak-over-mean across the four GPU partitions: 4.0× means one partition of four was doing anything at all.

THE CODE THAT PRODUCES IT

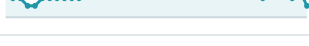
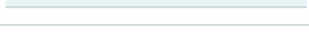
artifacts/.../baseline.log

JSON

```
1 "quadtreeRebuildCadenceSteps": 1,  
2 "structuredAirSupportRows": 1959,  
3 "structuredAirSupportCells": 4487,  
4 "structuredAirSupportFaceItems": 77352,  
5 "structuredAirSupportSeedFaces": 23916,  
6 "structuredAirSupportMarchDepth": 0,  
7 "rejectedAdvanceAttempts": 0,  
8 "volumeDrift": -3.77e-6
```

The candidate rebuild runs **every** advance — 240 rebuilds in 240 steps. The paper adapts per frame and books 8.6 % for it on the rotating-paddle scene; ours is 6.0 %, which is comparable and cheap for what it buys. The shape of the launches inside it is the problem, not their frequency.

STABILITY ACROSS THE TEN ADVANCES IN THE WINDOW

ENCODER	10 ADVANCES (own scale)	MEAN	MIN	MAX	REPR.
Octree resident grading closure		2.59	2.50	3.63	2.56
SPGrid V-cycle – candidate build level sets		2.14	2.09	2.23	2.16
Compare topology-tile refinement signatures		1.79	1.71	2.18	1.76
SPGrid V-cycle – candidate link parent chains		1.35	1.32	1.40	1.36
SPGrid V-cycle – candidate build level deltas		1.06	0.98	1.24	1.02

EVERY ENCODER IN S4, BY GPU TIME

#	ENCODER	MS	%	OCC	ALU	PLC
173	Octree resident grading closure	2.560	19.5	1.9	7.1	2.01
218	SPGrid V-cycle – candidate build level sets	2.160	16.4	0.6	0.2	4.00
170	Compare topology-tile refinement signatures	1.760	13.4	0.2	0.9	4.00
225	SPGrid V-cycle – candidate link parent chains	1.360	10.3	0.2	0.7	4.00
221	SPGrid V-cycle – candidate build level deltas	1.020	7.8	0.2	0.1	4.00
176	Classify exact dirty-tile frontier candidates	0.618	4.7	1.5	6.7	1.51
172	Octree reset and refinement	0.461	3.5	1.9	2.2	1.51
178	Sorted old/new frontier merge	0.416	3.2	0.3	0.7	3.00
233	SPGrid V-cycle – candidate publish spectral bounds	0.287	2.2	0	0	1.00
195	Finalize direct structured publication	0.232	1.8	0	26	1.00
197	Transfer accepted velocity to changed topology faces	0.180	1.4	5.9	3.9	1.17
232	SPGrid V-cycle – candidate build stencils	0.141	1.1	–	–	–
204	Rebuild symmetric structured boundary rows	0.124	0.9	–	–	–
180	Inactive octree pressure-row candidate	0.116	0.9	–	–	–
201	Resolve canonical free-surface boundary slots	0.112	0.9	–	–	–
177	Sort dirty frontier candidates by level and Morton	0.111	0.8	0.3	0.1	3.00
185	Resolve structurally dirty power descriptors	0.100	0.8	1.4	1.3	–
192	Prefix six structured velocity families	0.097	0.7	–	–	–
191	Classify direct structured catalog slots	0.096	0.7	–	–	–
168	Prepare inactive owner-page generation	0.092	0.7	–	–	–
226	SPGrid V-cycle – candidate mark brick occupancy	0.064	0.5	–	–	–
224	SPGrid V-cycle – candidate write transfers	0.055	0.4	–	–	–
200	Classify fine-over-coarse liquid rows	0.055	0.4	–	–	–
228	SPGrid V-cycle – candidate scatter ranked slots	0.050	0.4	–	–	–
220	SPGrid V-cycle – candidate insert ghosts	0.045	0.3	–	–	–
196	Reconstruct direct structured cell velocities	0.042	0.3	–	–	–
223	SPGrid V-cycle – candidate scan transfers	0.042	0.3	–	–	–
198	Reconstruct direct structured cell velocities	0.041	0.3	–	–	–
193	Scatter direct structured velocity slots	0.041	0.3	–	–	–
212	SPGrid V-cycle – capture reduce L1 delta	0.036	0.3	–	–	–
194	Publish direct Section 6.3 rows and worksets	0.033	0.3	–	–	–
217	SPGrid V-cycle – candidate clear levels	0.033	0.2	–	–	–
189	Stage exact power topology carry and affected rows	0.030	0.2	–	–	–
207	Structured boundary worksets – scan blocks	0.029	0.2	–	–	–
219	SPGrid V-cycle – candidate detect ghosts	0.029	0.2	–	–	–
186	Stage exact power descriptor carry and affected rows	0.028	0.2	–	–	–
179	Inactive octree pressure-row candidate dirty-row deterministic scan	0.028	0.2	–	–	–
182	Inactive octree pressure-row candidate row-delta compact publication	0.027	0.2	–	–	–
171	Build exact structural topology-tile delta	0.025	0.2	–	–	–
222	SPGrid V-cycle – candidate count transfers	0.025	0.2	–	–	–
188	Resolve affected power topology rows	0.025	0.2	–	–	–

#	ENCODER	MS	%	OCC	ALU	PLC
215	SPGrid V-cycle – candidate prepare live schedules	0.023	0.2	–	–	–
231	SPGrid V-cycle – candidate link page neighbours	0.018	0.1	–	–	–
187	Prepare power topology delta	0.018	0.1	–	–	–
183	Inactive octree pressure-row candidate row-delta validate publication	0.014	0.1	–	–	–
229	SPGrid V-cycle – candidate mark page occupancy	0.014	0.1	–	–	–
181	Inactive octree pressure-row candidate row-delta one-ring publication	0.014	0.1	–	–	–
211	SPGrid V-cycle – capture plan L1 delta	0.014	0.1	–	–	–
210	SPGrid V-cycle – select setup delta and capture changed L1	0.013	0.1	–	–	–
213	SPGrid V-cycle – build inactive exact level deltas	0.012	0.1	–	–	–
234	SPGrid V-cycle – candidate validate hierarchy	0.012	0.1	–	–	–
209	Structured boundary worksets – scatter families	0.012	0.1	–	–	–
235	Validate complete inactive topology epoch	0.012	0.1	–	–	–
208	Structured boundary worksets – scatter rows	0.012	0.1	–	–	–
206	Structured boundary worksets – count family classes	0.012	0.1	–	–	–
203	Commit canonical structured boundary slots	0.011	0.1	–	–	–
227	SPGrid V-cycle – candidate rank bricks	0.011	0.1	–	–	–
184	Prepare power descriptor control	0.011	0.1	–	–	–
174	Build exact wet-frontier tile delta	0.011	0.1	–	–	–
230	SPGrid V-cycle – candidate compact pages	0.011	0.1	–	–	–
216	SPGrid V-cycle – candidate commit changed L1	0.010	0.1	–	–	–
190	Begin direct structured publication	0.009	0.1	–	–	–
205	Structured boundary worksets – count row classes	0.009	0.1	–	–	–
199	Prepare structured boundary candidate-ready transaction	0.009	0.1	–	–	–
202	Resolve canonical solid boundary slots	0.009	0.1	–	–	–
175	Begin persistent octree leaf frontier	0.008	0.1	–	–	–
169	Commit inactive owner-page candidate bank	0.007	0.1	–	–	–
214	SPGrid V-cycle – candidate apply skip	0.005	0.0	–	–	–

The whole-device counters, and the four shapes that explain them

These are frame-wide totals. The per-band verdicts, the interactive mechanisms and the stability series all live in the eight bands — this section only names the shapes and says where each one is.

FRAME WALL

180.42 ms

The representative advance, anchor to anchor.

GPU BUSY

172.32 ms

95.5 % of wall. Idle is 8.10 ms across 441 gaps (mean 18 μ s, max 556 μ s).

MEAN COMPUTE OCCUPANCY

7.5 %

Per partition: 8.5 / 7.5 / 9.4 / 5.9 %.

MEAN ALU UTILISATION

14.5 %

Averaged over uncontended samples across the whole frame.

READ BANDWIDTH

2.64 GB/s

Instruments reports these counters already as a rate.

WRITE BANDWIDTH

0.44 GB/s

Aggregate **3.1 GB/s** — 3.9 % of this part's $\sim$ 78 GB/s observed peak.

The device is busy 96 % of the time and using about **12 % of its threads and 15 % of its ALUs** to do it. Aggregate memory traffic is **4.3 GB/s** — on a part whose observed streaming peak in this same profiler is $\sim$ 78 GB/s, that is **5.5 % of achievable bandwidth**.

For scale: the paper reports (Table 1, Intel Xeon E3-1241) a streaming-copy reference of 14.45 GB/s and their optimised second-order Laplacian sustaining **3.95 GB/s**. Our whole frame — on a GPU with an order of magnitude more bandwidth — moves data at roughly the rate their single-socket CPU kernel did. The bottleneck is not arithmetic intensity; it is that the work is decomposed into launches too small and too serial to fill the machine.

LAUNCH SHAPE	WHY IT COSTS	MS	WHERE IT HAPPENS
Thread-starved by construction	the launch cannot fill the machine before it executes an instruction	53.0	S3b
Launched wide, drained narrow	a provisioned-capacity grid over a work set that is orders smaller	88.1	S1 S3e S4
Memory / atomic bound, but well fed	high occupancy, near-zero ALU – the cost is write traffic	– retired	
Actually healthy	dense arithmetic, or a launch whose work was deleted rather than widened	31.5	S2 S3a S3c S3d

FIDELITY

Scorecard against the paper

The implementation is faithful. Where it departs it departs *upward*: both departures are items the paper's own Limitations section names as desirable future work.

PAPER ELEMENT	STATUS	EVIDENCE
§4.1 power-diagram discretization, eq (3)/(4)	IMPLEMENTED	7,756 power faces; 1,956 + 0 descriptor split
§4.1 embedded free-surface + solid boundaries	IMPLEMENTED	Resolve canonical free-surface / solid boundary slots , 0.13 ms
§4.2 pyramid of sparse uniform grids + ghost cells	IMPLEMENTED	6 levels; candidate detect / insert ghosts
§4.2 ghost-cell amendment for power diagrams	IMPLEMENTED	ghost detection consumes power-face adjacency
§4.3 $M = J^k(\text{band}) \rightarrow M_1$ V-cycle $\rightarrow J^k(\text{band})$, $k \approx 8$	IMPLEMENTED	$k = 8$, band 3 layers, 6-level first-order V-cycle
§4.3 satisfactory convergence in 6–10 PCG iterations	IMPROVED	4 iterations, rel. residual 7.10×10^{-5}
§5 narrow band at 4× resolution	IMPLEMENTED +	128×64×128; 1 packed u32/sample = 256 B/page. The paper's 4 channels became binary16 ϕ + 3 state bits + a 9-bit cached crossing; the marching channels are transient
§5 multi-stepped semi-Lagrangian, m = resolution factor	IMPLEMENTED	$m = 4$, Plan GPU-resident fine transport substeps
§5 cube trilinear / tetra barycentric interpolation	IMPLEMENTED	transitionSample + catalog tetrahedra
§6.2 point location inside the local tetrahedralization	IMPROVED	compiled to an adjacency walk with per-octant seeds (28.9 MB of derived tables) in both samplers; the paper searches. The fine-transport copy is unrolled to 11 straight-line steps against the graph's proven diameter 10, so its 64-lane workgroups have no data-dependent trip count
§5 allocate interface blocks + ring, then mark in parallel	IMPLEMENTED +	implemented with a 4-ring , not a 1-ring, and dilated as a separable X/Y/Z box sweep rather than a cubic scatter
§5 fine→coarse level-set correction	IMPLEMENTED	Fine-to-coarse restriction
§5 fast marching for redistancing	IMPROVED	replaced by parallel JFA closest-point transform – the paper's own future work
§5 velocity extrapolation on faces	IMPLEMENTED	77,352 face unknowns (6,446 rows × 12), run twice per advance
§6.1 19-bit topology encoding, two cases	IMPLEMENTED	1,956 / 0 split
§6.2 local Delaunay tetrahedralization lookup	IMPLEMENTED	8,083 configs, 68 max tetrahedra, precomputed offline (14.18 MB)
§6.3 hierarchical operator, 19 stored coefficients	IMPLEMENTED	catalog coefficients + GhostValueAccumulate
§5 first-order semi-Lagrangian for the level set	IMPLEMENTED	forward Euler per substep, as written. A BFECC ladder exists but coarsePhiBFECCEnabled gates it to fineFactor 1 + opt-in; it does not run on this lane
Volume conservation	ADDED	not in the paper; drift -3.77×10^{-6} over 299 steps (was -1.14×10^{-7} before binary16 ϕ)

CONCLUSIONS

The physics is not the cost

Every operator the paper spends its Section 4 and Section 6 on — the power-diagram Laplacian, the divergence, the gradient, the catalog lookup, the boundary coefficients — costs a combined **0.36 ms, 0.2 %** of the frame. Section 6.1's central bet — a compact topology descriptor plus a lookup table instead of explicit meshing — pays off completely.

THREE ITEMS ARE 72 % OF THE FRAME	MS/ADV (MEAN)	%	LIMITER
Sec 5 velocity extrapolation, two marches	65.9	36 %	12 barrier-separated waves; wide launch, narrow drain; 3-workgroup residual tail
Pressure solve	52.3	29 %	one workgroup, 256 threads, 0.2 % occupancy, ~355 serial barrier phases
Fine level-set advection	13.4	7 %	dense but divergent; the rare class is 63 % of it, and still scans the fan

The single most consequential number in this document is **0.2 %**. For 44 – 61 ms of every 180 ms frame, the M1 Max runs 256 of its 98,304 threads, pinned to one of 4 partitions, with the other 3 measuring 0.00 %. That is a deliberate, documented trade — bitwise determinism, which the D4 symmetry gate on this exact scene depends on, and which WGSL's lack of a device-wide barrier makes hard to buy any other way. But it is a trade, and at 1,959 unknowns per 52.3 ms (**37,493 unknowns/second**) it sets a hard ceiling on how large a scene this solver can carry: the executor's gate admits up to 65,536 rows, at a fixed 256-lane budget. It did not move this round, so it moved *up* the list — from 26 % of the frame to 29 %.

Second: 65.94 ms of every frame solves the same velocity-extrapolation problem twice. Whether the two publications can share carry state is an implementation question, not a numerical one — the paper needs the extended field to exist once per surface update, and books 2 – 5 % of its frame for it.

Third, and the lesson of this round: the wins came from deleting work, not from adding threads. Structured advection kept its 7.9 % residency ceiling and its 64-lane workgroups and still fell **10.6×** — because three nested searches became indexed reads, and because the one genuinely divergent branch was hoisted into its own uniform pass. The fine level-set page halved by storing ϕ at the precision the band actually needs and deleting cached neighbour pointers that a key can recompute. Neither change touched the discretization. The next two items on the list are the same shape: a second copy of the same search (item 3), and a second copy of the same extrapolation (item 2).

HEADROOM, ORDERED BY MEASURED MS – EACH ITEM IS WRITTEN OUT IN ITS BAND

#	CHANGE	≈ MS	BAND
1	Widen the pressure solve	52	S3b
2	Halve the Sec 5 marches	33	S1 S3e
3	Make the JFA fallback ladder conditional in fact, not just in form	8	S3d
4	Split the fine transport's common/rare classes on something that discriminates	3	S2
5	Fix the wide-launch / narrow-drain shape of the march waves	–	S1 S3e

Items 1 – 3 together account for ~94 ms of the 182.1 ms mean frame. Recovering most of that lands the frame near 112 ms [derived – arithmetic on the measured items, not an implemented result] without touching the discretization, the accuracy, or the symmetry guarantee.

STRUCK OFF – WHAT THE LIST HAS ACTUALLY RETURNED

CHANGE	SCORED	RETURNED	HOW
Give Advect structured families more threads	20	17.8	compiled point-location plus a uniform carry pass – S3a, band 3
Port the compiled sampler into fine transport	19	35.2	the same tables, unrolled rather than looped – S2, crosses the band change
Make the halo dilation hierarchical	8	8.1	separable rather than hierarchical, which is strictly better – S2

Three items scored at ≈47 ms returned 61.0 ms of encoder time — calibration for reading the estimates above, which are made the same way. The wall clock moved further than that sum because removing an encoder also removes its launch and drain: the three band-3 captures went 237.75 → 159.26 ms/advance, –78 ms.

The complete ordered frame

Every Metal compute encoder of the representative advance, in execution order. `t` is milliseconds from the frame's first encoder. `plc` is placement (peak-over-mean across the four GPU partitions): $1.0\times$ spread evenly, $4.0\times$ pinned to one quarter of the machine. `–` means no uncontended counter sample landed inside that encoder.

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
1	0.02	0.005	—	—	—	—	—	S1	Open coupled topology ready-commit gate
2	0.04	0.004	—	—	—	—	—	S1	Prepare exact accepted topology row commit
3	0.07	0.013	—	—	—	—	—	S1	Commit accepted topology row identities and pressure seed
4	0.11	0.008	—	—	—	—	—	S1	Publish ready owner-page generation
5	0.14	0.011	—	—	—	—	—	S1	Commit power descriptor publication
6	0.17	0.010	—	—	—	—	—	S1	Commit power topology publication
7	0.20	0.005	—	—	—	—	—	S1	Accept structured publication
8	0.23	0.005	—	—	—	—	—	S1	Accept structured boundary publication
9	0.26	1.650	0	0	1.4	1.4	4.00	S1	SPGrid V-cycle – publish validated exact level deltas
10	1.96	0.026	—	—	—	—	—	S1	Initialize structured air-support publication – topology-commit
11	2.02	0.993	36.7	23.1	16.6	0.1	1.09	S1	Publish structured air-support identities – topology-commit
12	3.06	0.924	6.9	10	2.9	0.1	—	S1	Close sparse fine air-support demand – topology-commit
13	4.03	0.039	—	—	—	—	—	S1	Emit sparse fine air-support demand – topology-commit
14	4.10	0.262	3.1	4.2	4.9	0.1	1.54	S1	Canonicalize footprint air-support identities – topology-commit
15	4.41	0.027	—	—	—	—	—	S1	Publish canonical air-support candidates – topology-commit
16	4.46	0.171	24.4	19.9	19.7	0.1	—	S1	Resolve and hash live adaptive air owners – topology-commit
17	4.69	0.491	2	2.6	15.8	0.1	—	S1	Extrapolate structured ordinary faces and reconstruct support vectors – topology-commit
18	5.23	32.106	9.9	22.3	3.2	0.3	1.43	S1	March Section 5 sparse changed frontier to a fixed point – topology-commit
19	37.39	0.917	6.4	11.7	15.4	6.3	1.55	S1	Reconstruct Section 5 air-support vectors – topology-commit
20	38.36	0.140	0.2	0.2	1.4	1.3	3.00	S2	Maintain compact octree fine seeds
21	38.56	0.066	—	—	—	—	—	S2	Publish deterministic fine-seed brick residency
22	38.66	0.184	—	—	—	—	—	S2	Plan GPU-resident fine transport substeps
23	38.90	0.049	5.7	16.5	7.8	2.6	1.04	S2	Classify direct structured fine transport blocks
24	38.97	0.017	—	—	—	—	—	S2	Reduce direct structured fine transport workset blocks
25	39.01	0.024	—	—	—	—	—	S2	Scan direct structured fine transport workset groups
26	39.06	0.007	—	—	—	—	—	S2	Publish direct structured fine transport worksets
27	39.09	0.007	—	—	—	—	—	S2	Compact direct structured fine transport worksets
28	39.13	2.900	14.3	27.1	0.1	0.1	1.15	S2	Advect fine phi common
29	42.08	10.560	15.4	37.9	0.2	0.1	1.02	S2	Advect fine phi rare
30	52.69	0.014	—	—	—	—	—	S2	Reduce structured fine transport status blocks
31	52.73	0.013	—	—	—	—	—	S2	Publish structured fine transport status
32	52.77	0.250	19	22.8	3.6	0.1	1.01	S2	Commit structured fine phi and phase delta
33	53.07	0.005	—	—	—	—	—	S2	Clear structured fine transport delta header
34	53.09	0.016	—	—	—	—	—	S2	Reduce structured fine transport delta blocks
35	53.13	0.017	—	—	—	—	—	S2	Publish structured fine transport delta
36	53.17	0.007	—	—	—	—	—	S2	Compact structured fine topology delta

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
37	53.20	0.006	—	—	—	—	—	S2	Clear exact global fine page delta
38	53.23	0.007	—	—	—	—	—	S2	Clear recurring fine-band identity mask
39	53.26	0.069	—	—	—	—	—	S2	Publish recurring sparse fine band (compact seed classification and rank)
40	53.38	0.008	—	—	—	—	—	S2	Seed recurring fine-band halos
41	54.01	0.012	—	—	—	—	—	S2	Dilate recurring fine-band halo axis 0
42	54.05	0.013	—	—	—	—	—	S2	Dilate recurring fine-band halo axis 1
43	54.09	0.015	—	—	—	—	—	S2	Dilate recurring fine-band halo axis 2
44	54.16	0.029	—	—	—	—	—	S2	Commit recurring fine-band halo
45	54.20	0.137	—	—	—	—	—	S2	Scan and compact recurring fine-band logical identity
46	54.35	0.038	—	—	—	—	—	S2	Classify exact global fine page delta
47	54.40	0.008	—	—	—	—	—	S2	Compact sorted global fine changed keys
48	54.42	0.007	—	—	—	—	—	S2	Prepare global fine page delta expansion
49	54.44	0.059	—	—	—	—	—	S2	Classify exact changed-key dirty and support pages
50	54.51	0.011	—	—	—	—	—	S2	Compact exact dirty and support pages
51	54.53	1.250	1.6	0.7	1.5	1.2	1.14	S2	Finalize global fine page delta
52	55.03	0.011	—	—	—	—	—	S2	Publish physical power-cell volumes
53	55.81	0.008	—	—	—	—	—	S2	Publish exact post-redistance fine summary keys
54	55.83	0.030	—	—	—	—	—	S2	Gather compact global fine flags/phi payloads
55	55.87	0.048	29.9	18.3	18.3	17.8	—	S2	Gather compact global fine work A/B payloads
56	55.93	0.027	—	—	—	—	—	S2	Snapshot exact global fine rollback pages
57	55.97	0.045	—	—	—	—	—	S2	Initialize added global fine samples
58	56.04	0.006	—	—	—	—	—	S2	Initialize added global fine work A/B samples
59	56.05	0.031	—	—	—	—	—	S2	Gather all compact global fine adjacency
60	56.09	0.006	—	—	—	—	—	S2	Finalize global fine publication
61	56.11	0.007	—	—	—	—	—	S2	Fine-to-coarse restriction – prepare
62	56.13	0.001	17.5	32.2	0.2	0.1	—	S2	Fine-to-coarse restriction – restrict
63	56.14	0.026	—	—	—	—	—	S2	Fine-to-coarse restriction – publish
64	56.18	0.016	—	—	—	—	—	S2	Fine JFA – publish support mask
65	56.19	0.007	—	—	—	—	—	S2	Power coarse level set – publish exact dispatch
66	56.21	0.006	—	—	—	—	—	S2	Fine JFA – reset recurring frontiers
67	56.23	0.777	—	—	—	—	—	S2	Fine JFA – refresh transported closest-point codes
68	56.24	0.025	0.4	0.9	2.7	1.2	2.53	S2	Power coarse level set – persistent schedule
69	56.29	0.041	53.9	74.1	5.5	0.1	1.00	S3a	Prepare accepted structured dynamics worksets
70	56.37	0.586	40.3	62.9	2.8	1	1.01	S3a	Flatten structured boundary carry probes
71	57.00	2.620	3.3	6.9	3.8	1.8	1.44	S3a	Advect structured families
72	59.67	0.016	—	—	—	—	—	S3a	Commit advected structured families
73	59.70	0.022	53.9	74.1	5.5	0.1	1.00	S3a	Prepare accepted structured dynamics worksets
74	59.75	0.019	—	—	—	—	—	S3a	Force and constrain structured families
75	59.80	0.031	—	—	—	—	—	S3a	Fuse structured divergence RHS rows
76	59.87	0.006	—	—	—	—	—	S3a	Zero dry-identity divergence RHS rows

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
77	59.91	53.020	0.2	0.2	1.5	0.1	4.00	S3b	Octree persistent MGPCG – whole solve in one workgroup
78	113.00	0.023	53.9	74.1	5.5	0.1	1.00	S3c	Prepare accepted structured dynamics worksets
79	113.05	0.023	–	–	–	–	–	S3c	Mark structured overhead separation rows
80	113.10	0.017	–	–	–	–	–	S3c	Project structured families
81	113.15	0.053	–	–	–	–	–	S3c	Reconstruct projected structured rows
82	113.28	0.002	–	–	–	–	–	S3d	Fine JFA – clear recurring frontier masks
83	113.31	0.002	–	–	–	–	–	S3d	Fine JFA – mark recurring dirty frontier
84	113.35	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 6
85	113.38	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 5
86	113.40	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 4
87	113.42	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 3
88	113.45	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 2
89	113.47	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 1
90	113.50	0.002	–	–	–	–	–	S3d	Fine JFA – build recurring frontier 0
91	113.52	0.005	–	–	–	–	–	S3d	Fine JFA – finalize recurring frontier dispatches
92	113.56	0.458	16.6	17.2	4.3	3.7	–	S3d	Fine JFA – seed closest points
93	114.06	0.002	–	–	–	–	–	S3d	Fine JFA – cooperative flood A to B stride 8
94	114.09	0.002	–	–	–	–	–	S3d	Fine JFA – cooperative flood B to A stride 4
95	114.12	1.040	43.8	72.3	4.1	0.4	1.11	S3d	Fine JFA – cooperative flood A to B stride 2
96	114.56	0.010	–	–	–	–	–	S3d	Reset global volume reduction
97	115.21	1.010	53.3	72.9	4.4	0.7	1.00	S3d	Fine JFA – cooperative flood B to A stride 1
98	116.27	1.010	53.4	72.9	4.6	0.7	1.01	S3d	Fine JFA – cooperative flood A to B stride 1
99	117.34	1.010	53.3	72.9	4.4	0.7	1.00	S3d	Fine JFA – cooperative flood B to A stride 1
100	118.39	0.009	–	–	–	–	–	S3d	Prepare exact compact coarse volume dispatch
101	118.42	0.011	–	–	–	–	–	S3d	Reduce compact coarse volume partials
102	118.45	0.270	–	–	–	–	–	S3d	Fine JFA – resolve A to B
103	118.45	0.015	–	–	–	–	–	S3d	Finalize compact coarse volume
104	118.49	0.014	–	–	–	–	–	S3d	Prepare active global fine volume dispatch
105	118.77	0.032	–	–	–	–	–	S3d	Fine JFA – prepare warm fallback
106	118.83	0.001	–	–	–	–	–	S3d	Fine JFA – refresh full-support fallback closest-point codes
107	118.86	0.002	–	–	–	–	–	S3d	Fine JFA – seed full-support fallback closest points
108	118.88	0.001	–	–	–	–	–	S3d	Fine JFA – complete support fallback 1 stride 8
109	118.91	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback 2 stride 4
110	118.93	0.001	–	–	–	–	–	S3d	Fine JFA – complete support fallback 3 stride 2
111	118.96	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback 4 stride 1
112	118.98	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback 5 stride 1
113	119.00	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback 6 stride 1
114	119.03	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback collar repair 3
115	119.05	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback collar repair 4

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
116	119.08	0.002	–	–	–	–	–	S3d	Fine JFA – complete support fallback collar repair 5
117	119.10	0.002	–	–	–	–	–	S3d	Fine JFA – resolve complete support fallback
118	119.12	0.029	–	–	–	–	–	S3d	Fine JFA – finalize
119	119.18	0.026	–	–	–	–	–	S3d	Fine JFA – commit distances
120	119.29	1.360	16.6	65.8	0.6	0.1	1.00	S3d	Reduce resident fine overlap partials
121	120.70	0.044	–	–	–	–	–	S3d	Finalize global fine volume
122	120.79	0.026	–	–	–	–	–	S3d	Apply bounded global fine normal correction
123	120.84	1.420	11.1	43.7	0.6	0.1	1.33	S3d	Re-reduce corrected global fine volume
124	122.32	0.040	–	–	–	–	–	S3d	Finalize first corrected global fine volume
125	122.40	0.027	–	–	–	–	–	S3d	Apply residual bounded global fine normal correction
126	122.45	1.350	16.7	64.9	0.6	0.1	1.00	S3d	Measure twice-corrected global fine volume
127	123.86	0.040	0	0	0	0.1	1.00	S3d	Finalize measured global fine volume
128	123.93	0.009	–	–	–	–	–	S3d	Finalize global fine publication
129	123.97	0.022	–	–	–	–	–	S3d	Settle deferred global fine publication
130	124.03	0.015	–	–	–	–	–	S3d	Settle deferred rejected fine work payload
131	124.08	0.009	–	–	–	–	–	S3d	Fine-to-coarse restriction – prepare
132	124.11	0.269	17.5	32.2	0.2	0.1	–	S3d	Fine-to-coarse restriction – restrict
133	124.44	0.027	–	–	–	–	–	S3d	Fine-to-coarse restriction – publish
134	124.49	0.161	–	–	–	–	–	S3d	Power coarse level set – publish exact dispatch
135	124.68	0.376	0.4	0.9	2.7	1.2	2.53	S3d	Power coarse level set – persistent schedule
136	125.08	0.013	–	–	–	–	–	S3d	Prepare direct fine-summary publication
137	125.12	0.007	–	–	–	–	–	S3d	Validate exact fine-summary delta
138	125.16	0.010	–	–	–	–	–	S3d	Validate fine-summary coarse rows
139	125.20	0.014	0	0	0	0.1	1.00	S3d	Retire direct coarse summary ranks
140	125.24	0.022	–	–	–	–	–	S3d	Remove direct fine-summary page references
141	125.29	0.005	–	–	–	–	–	S3d	Prepare sparse fine-summary directory-page reclamation
142	125.32	0.009	–	–	–	–	–	S3d	Reclaim empty fine-summary directory pages
143	125.36	0.092	–	–	–	–	–	S3d	Ensure sparse fine-summary directory pages
144	125.47	0.013	–	–	–	–	–	S3d	Ensure sparse coarse-summary directory pages
145	125.51	0.041	–	–	–	–	–	S3d	Ensure compact fine-summary ranks
146	125.58	0.012	–	–	–	–	–	S3d	Ensure compact coarse-summary ranks
147	125.62	0.015	–	–	–	–	–	S3d	Add direct fine-summary page references
148	125.66	0.012	0	0	0	0.1	1.00	S3d	Publish direct coarse summary ranks
149	125.70	0.006	–	–	–	–	–	S3d	Prepare active fine-summary mip dispatch
150	125.72	0.148	–	–	–	–	–	S3d	Recompute direct fine-summary bases
151	125.75	0.006	–	–	–	–	–	S3d	Stamp octree topology attempt generation
152	125.89	0.043	–	–	–	–	–	S3d	Recompute direct fine-summary parents level 1
153	125.97	0.036	–	–	–	–	–	S3d	Recompute direct fine-summary parents level 2
154	126.03	0.039	–	–	–	–	–	S3d	Recompute direct fine-summary parents level 3
155	126.11	0.043	–	–	–	–	–	S3d	Recompute direct fine-summary parents level 4

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
191	171.71	0.096	–	–	–	–	–	S4	Classify direct structured catalog slots
192	171.86	0.097	–	–	–	–	–	S4	Prefix six structured velocity families
193	172.00	0.041	–	–	–	–	–	S4	Scatter direct structured velocity slots
194	172.06	0.033	–	–	–	–	–	S4	Publish direct Section 6.3 rows and worksets
195	172.13	0.232	0	26	14.9	3.9	1.00	S4	Finalize direct structured publication
196	172.41	0.042	–	–	–	–	–	S4	Reconstruct direct structured cell velocities
197	172.50	0.180	5.9	3.9	0.9	0.1	1.17	S4	Transfer accepted velocity to changed topology faces
198	172.73	0.041	–	–	–	–	–	S4	Reconstruct direct structured cell velocities
199	172.82	0.009	–	–	–	–	–	S4	Prepare structured boundary candidate-ready transaction
200	172.85	0.055	–	–	–	–	–	S4	Classify fine-over-coarse liquid rows
201	172.94	0.112	–	–	–	–	–	S4	Resolve canonical free-surface boundary slots
202	173.10	0.009	–	–	–	–	–	S4	Resolve canonical solid boundary slots
203	173.14	0.011	–	–	–	–	–	S4	Commit canonical structured boundary slots
204	173.17	0.124	–	–	–	–	–	S4	Rebuild symmetric structured boundary rows
205	173.35	0.009	–	–	–	–	–	S4	Structured boundary worksets – count row classes
206	173.38	0.012	–	–	–	–	–	S4	Structured boundary worksets – count family classes
207	173.41	0.029	–	–	–	–	–	S4	Structured boundary worksets – scan blocks
208	173.47	0.012	–	–	–	–	–	S4	Structured boundary worksets – scatter rows
209	173.50	0.012	–	–	–	–	–	S4	Structured boundary worksets – scatter families
210	173.53	0.013	–	–	–	–	–	S4	SPGrid V-cycle – select setup delta and capture changed L1
211	173.57	0.014	–	–	–	–	–	S4	SPGrid V-cycle – capture plan L1 delta
212	174.07	0.036	–	–	–	–	–	S4	SPGrid V-cycle – capture reduce L1 delta
213	174.12	0.012	–	–	–	–	–	S4	SPGrid V-cycle – build inactive exact level deltas
214	174.19	0.005	–	–	–	–	–	S4	SPGrid V-cycle – candidate apply skip
215	174.23	0.023	–	–	–	–	–	S4	SPGrid V-cycle – candidate prepare live schedules
216	174.28	0.010	–	–	–	–	–	S4	SPGrid V-cycle – candidate commit changed L1
217	174.35	0.033	–	–	–	–	–	S4	SPGrid V-cycle – candidate clear levels
218	174.41	2.160	0.6	0.2	0	0.1	4.00	S4	SPGrid V-cycle – candidate build level sets
219	176.62	0.029	–	–	–	–	–	S4	SPGrid V-cycle – candidate detect ghosts
220	176.67	0.045	–	–	–	–	–	S4	SPGrid V-cycle – candidate insert ghosts
221	176.76	1.020	0.2	0.1	0.1	0.1	4.00	S4	SPGrid V-cycle – candidate build level deltas
222	177.83	0.025	–	–	–	–	–	S4	SPGrid V-cycle – candidate count transfers
223	177.88	0.042	–	–	–	–	–	S4	SPGrid V-cycle – candidate scan transfers
224	177.95	0.055	–	–	–	–	–	S4	SPGrid V-cycle – candidate write transfers
225	178.05	1.360	0.2	0.7	0	0.1	4.00	S4	SPGrid V-cycle – candidate link parent chains
226	179.47	0.064	–	–	–	–	–	S4	SPGrid V-cycle – candidate mark brick occupancy
227	179.58	0.011	–	–	–	–	–	S4	SPGrid V-cycle – candidate rank bricks
228	179.62	0.050	–	–	–	–	–	S4	SPGrid V-cycle – candidate scatter ranked slots
229	179.72	0.014	–	–	–	–	–	S4	SPGrid V-cycle – candidate mark page occupancy

#	T	MS	OCC %	ALU %	RD GB/S	WR GB/S	PLC	BAND	ENCODER
230	179.76	0.011	—	—	—	—	—	S4	SPGrid V-cycle – candidate compact pages
231	179.79	0.018	—	—	—	—	—	S4	SPGrid V-cycle – candidate link page neighbours
232	179.83	0.141	—	—	—	—	—	S4	SPGrid V-cycle – candidate build stencils
233	180.02	0.287	0	0	0	0.1	1.00	S4	SPGrid V-cycle – candidate publish spectral bounds
234	180.34	0.012	—	—	—	—	—	S4	SPGrid V-cycle – candidate validate hierarchy
235	180.37	0.012	—	—	—	—	—	S4	Validate complete inactive topology epoch

APPENDIX B – REPRODUCTION

```
node --import tsx tools/profile-mini-dam-xctrace.ts --lane=symmetric-expansion \
--steps=240 --counter-seconds=8 --out=artifacts/xctrace-symmetric-expansion-2026-08-04-compiled
```

~20 minutes: a clean untraced baseline, then a relaunch with `use_user_defined_labels_in_backend` and an attached Metal System Trace + GPU-counter window in steady state. Outputs `report.html`, `summary.json` (every number on this page), the raw NDJSON tables and the `.trace` bundle. Solver-side facts come from `baseline.log` records `phase:"constructed"`, `phase:"final-performance-authority"` and `phase:"result"`.

The `--counter-seconds=8` is load-bearing. The default 3 s window produced a capture whose Metal *encoder metadata* stream was empty — GPU intervals were recorded, but nothing could be attributed to a pass, and the tool aborted rather than emit a composite report. The longer window also makes the harness extend the run: it stepped 299 advances rather than the 240 requested, because an 8 s counter window plus Instruments' ~2.7 s attach does not fit inside a shorter stepping phase. The wall-clock control in the lineage above is the 240-advance untraced baseline either way.

The scene's correctness gate now matches this profile. `c8c84ee` made interface band 4 with fine factor 4 canonical for `symmetric-expansion`, so `npm run test:webgpu:symmetric-expansion` exercises the same discretization this document profiles. It previously ran coarse-only (`FLUID_OCTREE_GLOBAL_FINE_FACTOR=1`), with no S2 or S3d in its frame at all.